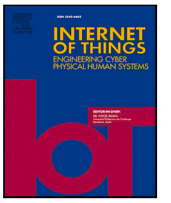
Internet of Things 22 (2023) 100819

Contents lists available at ScienceDirect

Internet of Things

journal homepage: www.elsevier.com/locate/iot



Review article

# Intrusion detection system for large-scale IoT NetFlow networks using machine learning with modified Arithmetic Optimization Algorithm

Salam Fraihat [a,*], Sharif Makhadmeh [a], Mohammed Awad [b], Mohammed Azmi Al-Betar [a], Anessa Al-Redhaei [a]

[a] *Artificial Intelligence Research Center (AIRC), College of Engineering and Information Technology, Ajman University, P.O. Box 346, Ajman, United Arab Emirates*

[b] *Department of Computer Science and Engineering, American University of Ras Al Khaimah, P.O. Box 72603, Ras Al Khaimah, United Arab Emirates*

ARTICLE INFO

*Keywords:*
Intrusion detection system
NetFlow
Internet of Things
Arithmetic optimization algorithm
Machine learning
Prediction
Classification
Feature selection
Optimization

ABSTRACT

With the rapid expansion of Internet of Things (IoT) networks, the need for robust security measures to detect and report potential threats is becoming more urgent. In this paper, we propose a Network Intrusion Detection System (NIDS), as a security measure, for large-scale IoT NetFlow-based networks. The proposed NIDS employs Machine Learning (ML) boosted by a modified version of the Arithmetic Optimization Algorithm (AOA) to determine the most suitable set of features. The selected seven features are used to train several ML models, including Random Forest and Extra Trees. Our research utilized four large datasets, released in 2021, containing IoT traffic data represented in a standard set of 43 NetFlow-based features. Reducing the number of features from 43 to 7 enhanced the prediction time and, consequently, the performance in the real world. Interestingly, the proposed NIDS exhibited a very accurate and robust detection model for IoT NetFlow data, which can be generalized for other Intrusion Detection datasets. Our proposed NIDS achieved up to 99% and 98% accuracy for binary and multi-classification, respectively. These scores were similar to those achieved by the state-of-the-art systems despite decreasing the number of utilized features by up to 84%.

## 1. Introduction

The networks of the Internet of Things (IoT) consist of a massive number of systems and sensors connected over the Internet. According to the International Data Corporation (IDC), by 2025, the number of IoT devices connected to the Internet is expected to exceed 41 billion [1]. This massive connectivity facilitated technological advancement in various domains, including smart cities, health care, and energy management [2]. However, due to the sensitive nature of the exchanged data, there have been increasing concerns about the privacy and security of such data. IoT networks comprise a fertile environment for attackers to launch various attacks [3]. Such cyberattacks may compromise a system's confidentiality, integrity, and availability.

An efficient detection and prevention system is necessary, with billions of connected devices and zettabytes of data exchanged across IoT networks. Network Intrusion Detection Systems (NIDS) are crucial in protecting networks from cyberattacks by monitoring the traffic data [4]. NIDS can generally be classified into signature or anomaly-based tools. Signature-based detection systems

* Corresponding author.
*E-mail address:* s.fraihat@ajman.ac.ae (S. Fraihat).

https://doi.org/10.1016/j.iot.2023.100819
Received 19 February 2023; Received in revised form 26 April 2023; Accepted 12 May 2023
Available online 16 May 2023

rely on and compare incoming data against lists of pre-existing attacks [5]. On the other hand, anomaly-based systems utilize artificial intelligence (AI) to identify patterns and flag cyberattacks, which is advantageous when detecting zero-day attacks [6]. However, like any system that relies on AI, anomaly-based NIDS is susceptible to false positives, and the system's accuracy cannot be 100% guaranteed. Thus, a hybrid system would combine the advantages of both approaches and overcome the mentioned shortages [7]. An anomaly-based NIDS may utilize various artificial intelligence techniques to classify network traffic. For example, supervised or unsupervised methods, including neural networks, deep learning, and dynamic clustering, have been employed to detect intrusions [8].

Realistic network data is challenging to obtain. Thus, many researchers created their own networks and launched attacks to generate data traffic, which was later used to design detection systems. Consequently, many of the existing detection systems are impractical and dataset dependent. Additionally, this approach resulted in various datasets with different feature sets making it difficult to test a specific NIDS across several networks [9]. In 2021, to overcome this problem, Sarhan et al. converted four widely used NIDS datasets (UNSW-NB15, BoT-IoT, ToN-IoT, and CSE-CIC-IDS2018) to NetFlow-featured datasets. The new datasets were named NF-UNSW-NB15-v2, NF-BoT-IoT-v2, NF-ToN-IoT-v2, and NF-CSE-CIC-IDS2018-v2. The four datasets share the same 43 NetFlow (NF) extracted features [9]. Notice that each NF version has the same dataset's attack types but consists of NF-based features. The transformation was achieved by converting the original packet capture files into NF format using the nprobe tool. Unlike most recent work utilizing outdated datasets, we will evaluate our model using these four newly created features.

NetFlow is a network protocol released by Cisco to collect and monitor traffic data [10]. NetFlow contains information related to network traffic, such as the incoming number of packets, packet sizes, port numbers, and stream duration. Testing an intrusion detection system across several datasets assures the system's scalability and adaptability to real-world scenarios. Out of the 43 NetFlow features, Sarhan et al. dropped some identifiers to avoid bias towards a specific source or destination. These four features were related to the IP addresses and port numbers of the source and destination. In their work, the authors have shown that NetFlow-extracted features (such as the protocol, incoming and outgoing bytes) contain patterns that can be used to predict attacks successfully. For example, their Extra Trees classifier achieved high accuracy across the four new datasets utilizing the 39 NF features. In fact, their model achieved higher binary classification accuracy for all new datasets compared to their original counterparts [9]. Other researchers utilized NetFlow properties to detect network anomalies. For example, Leslie developed a semi-supervised learning model that uses the NetFlow (regular and background) files in a dataset called CTU-13. The proposed system predicted botnet traffic with higher accuracy than that obtained on the original dataset [11].

A standard feature set across all datasets is an outstanding concept; however, we believe that the number of features can be minimized. Minimizing the number of features will enhance the time and space complexity. A lower number of features implies lower computational cost and faster prediction time. It also means less storage space [12]. In other words, the lower the number of utilized features, the faster the prediction time. Needless to say, it is vital to utilize such resources in restricted IoT environments efficiently. Of course, a feature's importance varies from dataset to dataset. Thus, such a process entails defining a subset of suitable features. In order for this subset to qualify as a new standard, it should achieve high accuracy across several datasets.

Several traditional feature selection (FS) methods exist to determine the most representative features for a whole dataset [13]. However, quite recently, the feature selection problem has been modeled as an optimization problem to find the minimum number of relevant features representing the whole set of data. Therefore, optimization algorithms have been used as feature selection methods for NIDS. For example, a NIDS was developed using Support Vector Machine (SVM) with Grey Wolf Optimizer (GWO) as a feature selection method [14,15]. Harmony search algorithms have also been used in the intrusion detection domain [16]. Indeed, many optimization methods have been utilized in FS to improve cybersecurity. Since the FS problem has a binary domain, the optimization methods can be enhanced to deal with the binary aspect. Although many optimization methods were utilized for NIDSs, no superior algorithm can find the best subset of features for all FS problems, per the No Free Lunch Theorem [17]. Thus, there is still an opportunity to utilize other optimization algorithms to develop NIDS.

Recently, inspired by the four basic arithmetic operations in math (multiplication, division, subtraction, and addition), the Arithmetic Optimization Algorithm (AOA) was proposed [18]. AOA has several advantages over other optimization problems, such as simplicity, ease of use, adaptability, flexibility, admissibility, soundness, and completeness. Furthermore, it has a great feature in its ability to deeply exploit the problem search space regions to which it converges, exploring several search space regions simultaneously. Thus, it strikes the right balance between exploration and exploitation during the search. Therefore, it has been successfully implemented to tackle a wide range of optimization problems within a short period, including engineering design problems [19], simple feature selection problems [20], IoT workflow scheduling problems [21], and many others.

This research work aims to:

- Provide an overview of the problem of intrusion detection in large-scale IoT NetFlow networks
- Offer a detailed literature review of the existing methods and their limitations
- Propose and evaluate a novel IoT intrusion detection system

This paper proposes a Network Intrusion Detection System (NIDS) for a large-scale IoT network. The system utilizes Machine Learning (ML) boosted by a modified version of the Arithmetic Optimization Algorithm (AOA). As mentioned earlier, ML proved advantageous in detecting unprecedented attacks, and optimization was deemed beneficial in deciding the most important features. The proposed model is designed as an efficient method for NetFlow-based networks where it has been constructed in three consecutive phases:

1. In the initial phase, the preprocessing stage is adapted, and incomplete, incorrect, duplicates, and irrelevant features and samples are removed. Furthermore, the data labeling method is applied to transform the categorical data. Then, prior to the features selection phase, the data undersampling method, based on proportional stratified sampling, is employed to degrade the size of NetFlow data.
2. The second phase is the feature selection phase, which uses the data prepared in the preprocessing stage. In this phase, four widely used optimization algorithms were employed to select the best features representing attack class patterns. The four algorithms were the Arithmetic Optimization Algorithm (AOA) [18], the White Shark Optimizer (WSO) [22], the Grey Wolf Optimizer (GWO) [23], and the Bat algorithm (BAT) [24]. Subsequently, a modified version of AOA was used as a feature selection method to find a subset of relevant features to be used by the Decision Tree model.
3. In the final phase, the subset of features obtained by the modified AOA was used in several ML models such as Naive Bayes (NB) [25], Random Forest [26] (RF), Decision Tree (DT) [27], Extra-Tree (ET) [9], and eXtreme Gradient Boosting (XGB) [28], where the results are recorded in terms of Accuracy, Precision, Detection Rate, F1-score, and False Acceptance Rate for both binary and multi-class classification.

The proposed model is evaluated from different perspectives, including the suitability of the optimization algorithms and ML methods used. The system was evaluated using four large datasets. The optimization results are evaluated using several measurements: error rate, number of features, and accuracy. Interestingly, the proposed model's reported accuracy exceeds 98%.

The rest of the paper is structured as follows: Section 2 offers the analysis of the literature review. Section 3 presents the methodology, including background about the base method used and the research stages of the proposed method. The experimental results and discussion of the results obtained are summarized in Section 4. Finally, Section 5 concludes the paper and provides windows for possible future directions.

## 2. Related work

As mentioned in the previous section, our proposed system utilized four recently created datasets. These datasets contain millions of network traffic data in Cisco's NetFlow protocol. Furthermore, our NIDS employs optimization to determine the most suitable features.

This section reports on systems that utilize NetFlow-based features to detect attacks. Then, it reviews the role of optimization in intrusion detection.

### *2.1. NetFlow-based IDSs*

Feature importance has been thoroughly investigated in the literature. In a recent study [12], the authors experimented with six different datasets, three of which were NetFlow-based (NF-UNSW-NB15, NF-CSE-CIC-IDS2018, and NF-ToN-IoT). In their work, they applied three feature selection techniques, namely Chi-Square, Information Gain, and Correlation, to sort the dataset's features by importance. The NetFlow-based datasets consisted of the same eight features: PROTOCOL, L7_PROTO, TCP_FLAGS, FLOW_DURATION, IN_BYTES, OUT_BYTES, IN_PKTS, and OUT_PKTS. Interestingly, each selection technique produced a different feature order. Also, the order was different for the same technique over other datasets. The authors used Deep Feed Forward (DFF) and Random Forest (RF) classifiers to test each feature subset [12].

Their paper reported the possibility of achieving high detection performance with a small subset of features. However, they also concluded that due to the variance in feature importance ranking from one dataset to the other, there is no general rule to decide on the optimal feature set and size. In other words, no specific set of features consistently achieved high detection rates. However, the use of all eight NetFlow features consistently achieved promising results. For example, the best obtained F1-scores for the NF datasets were 0.84, 0.85, and 1.00 using RF over NF-CSE-CIC-IDS2018, NF-UNSW-NB15, and NF-ToN-IoT, respectively along with of 95.51%, 98.50%, and 99.38% accuracy [12].

A common observation in their experiments was the reduction in prediction time when minimizing the number of features. Another interesting finding was the detected bias produced by Time-to-Live (TTL) based features in UNSW-NB15 dataset. Their experiments have shown that the classifier has achieved a maximum prediction performance by using a single TTL-based feature (such as sttl). The unrealistic high-importance rank of the TTL features is most likely due to the testbed design of this dataset. Thus, the inclusion of such features may result in biased results [12]. Indeed, the three UNSW-NB15 TTL-based features (sttl, dttl, and ct_state_ttl) have shown an abnormal predictive capability. However, none of these features are part of NF-UNSW-NB15-v2. Furthermore, our research found that TTLbased features alone, such as MAX_TTL, had little predictive power.

Karanfilovska et al. [29] applied several supervised and unsupervised machine learning techniques to detect anomalies. In their work, the authors worked on 10% of NF-ToN-IoT-v2 and achieved accuracy of 98.6% using Random Forest and 98.8% using XGBoost. The reported F1-score was identical to the accuracy in both models. However, the experiments were conducted using all 43 dataset features without dropping any identifier. Thus, the results were biased.

The authors of [30] applied several feature selection techniques on five NetFlow-based datasets, namely Chi-Square, Random Forest Importance, and Lasso L1. After that, ten (10) features were chosen to conduct binary classification across the datasets. Four ML models were used. However, Random Forest produced the best detection results across the five datasets. The authors chose the following ten features: FLOW_DURATION_MILLISECONDS, TCP_WIN_MAX_IN, DURATION_OUT, MAX_TTL, L7_PROTO, SRC_TO_DST_AVG_THROUGHPUT, SHORTEST_FLOW_PKT, MIN_IP_PKT_LEN, TCP_WIN_MAX_OUT, and OUT_BYTES. The selected

features do not include identifiers or hidden labels, and the Random Forest classifier produced excellent binary classification accuracy of around 100%. However, the authors did not refer to multiclassification in their paper. Furthermore, the authors seem to only have utilized extremely sampled and balanced subsets of the datasets in their experiments. The authors mentioned that "50% of each subset consists of infected traffic, and 50% of clean traffic" in their work. For example, NF-BoT-IoT-v2 consists of 99.64% (37,628,460) attack and 0.36% (135,037) benign samples. Thus, utilizing 30% (11,85,8347 records) of the dataset and generating over 5 million additional benign samples alters the dataset significantly, which may affect the model's prediction capability. In our work, we trained our classifiers on the all datasets and tested our features capability in classifying both binary and multi-classification.

The authors of [31] examined the importance of NF-ToN-IoT-v2 NetFlow-based features. The authors implemented several ML models, including Decision Trees and Random Forest, which achieved the highest results. The authors concluded that using features with values above the mean value of the 43 features combined is sufficient to classify network traffic. Thus, they proposed subsets of 13 and 17 features for binary and multi-classification, respectively. These subsets resulted in F1-scores of 1.00 in binary and 0.98 in multi-classification. While the proposed feature subsets are significantly smaller than the originals, we believe they can be reduced further. Additionally, the proposed features were tested on one dataset.

Sarhan et al. have shown interest in creating a standard set of features to be used across several NIDS datasets [9]. A common set of features allows researchers to thoroughly test and evaluate their systems across various datasets, ultimately bridging the gap between NIDS research and real-world NIDS deployment. In their quest, Sarhan et al. extracted 43 NetFlow-based features to recreate four well-known datasets, namely UNSW-NB15 [32], BoT-IoT [33], ToN-IoT [34], and CSE-CIC-IDS2018 [35]. The new datasets have been named NF-UNSW-NB15-v2, NF-BoT-IoT-v2, NF-ToN-IoT-v2, and NF-CSE-CIC-IDS2018-v2 [9]. The authors have shown that these 43 NetFlow-based features provide sufficient input to classify binary and multi-class attacks accurately [9]. The 2021 NetFlow-based datasets supported NIDS research and provided a fertile research environment. However, we argue that investigating 43 features may affect prediction speed, especially over IoT networks where computational and storage resources are constrained. Thus, a smaller subset of features with similar performance would be advantageous. The authors dropped a few identifiers in their work to avoid learning bias. For example, the IPs' and ports' source and distention features were excluded. In our paper, we propose seven features that perform similarly to those obtained using the 43 NF features. A detailed comparison between both feature sets is presented later in the paper. Additionally, the same authors [36] also applied several ML models across three different datasets using different feature selection methods. Their experiments inferred that no superior feature selection method or ML model exists across all datasets. This finding sparked our interest in examining and evaluating all possible feature subsets over several datasets.

Sayed et al. [37] developed two Convolutional Neural Networks (CNNs) to multi-classify IoT attacks in the NF-UNSW-NB15-v2 dataset. Both models (IoTCNN and MyCNN) have resulted in high accuracy with slightly better performance by MyCNN. The NF-UNSW-NB15-v2 data was converted to Red, Green, and black (RGB) images, which were later on fed to the CNN models. Out of the 43 NF features, the authors utilized 37 in their models. However, it is unclear whether any identifying features were included in the selected 37. Due to performance reasons, only 40% of the dataset was used. NF-UNSW-NB15-v2 contains less than 4% of attack samples. The authors mentioned that the imbalanced dataset introduced some limitations on the models' performance, such as prediction speed. However, the real-world traffic is most likely imbalanced. In their work, the authors did not refer to binary classification.

Le et al. applied DT and RF on portions of NF-BoT-IoT-v2 and NF-ToN-IoT-v2 and used SHapley additive exPlanations (SHAP) to explain the results obtained by both classifiers. Understanding how the models made their decisions and based on which features is highly beneficial to cybersecurity experts who would frequently analyze and evaluate such decisions [38]. Similarly, the authors of [39] utilized SHAP to explain how and why IDS features are selected by various ML models, including Convolutional Neural Networks (CNN).

The authors of [40] utilized SHAP to identify the most important features and explain their models' prediction results. The authors applied RF and DFF models over NF-BoT-IoT-v2, NF-ToN-IoT-v2, and NF-CSE-CIC-IDS2018-v2 datasets. In their work, they analyzed the top 20 influencing features for each classifier (RF and DFF) over the three datasets. Their findings showed a significant feature overlap between both classifiers. For example, 17 of NF-ToN-IoT-v2 top 20 features were common between the RF and DFF models. The overlap was slightly lower in the other two datasets (13 and 14).

### *2.2. Optimization methods for NIDS*

A plethora of research has been conducted to develop NIDSs employing ML models boosted by optimization methods. Below, we summarize a few NIDSs that utilize recently published optimization methods.

The hyperparameters of the Long short-term memory (LSTM) model are optimized by the Enhanced Coyote Optimization algorithm for Intrusion Detection System (IDS) in cloud environment [41]. Their model was tested using benchmark datasets where it excelled in terms of detection rate. In another study, the hybrid metaheuristic optimization techniques were proposed as FS methods for NIDS [42]. Their system hybridized Grey Wolf Optimizer (GWO) and Dipper Throated Optimizer (DTO) in an IoT environment. The proposed method is implemented against IoT-IDS dataset and revealed very successful outcomes compared with other state-of-the-art NIDSs.

Alkanhel et al. proposed a NIDS based on GWO and DTO and applied it to locality-sensitive hashing and synthetic minority oversampling techniques [43]. The experiments were conducted using seven different types of attack cases where the proposed model was able to achieve very competitive detection accuracy (98.1%), sensitivity (97.8%), and specificity (98.8%). In [44], the

Whale Optimization Algorithm (WOA) and DTO were hybridized to boost the voting classifier performance for NIDS. The authors created new IoT datasets for evaluation purposes, and the results revealed the superiority of their proposed NIDS.

Vanitha et al. [45] improved the Ant Colony Optimization (ACO) to boost the performance of different ML models, such as Distance Decision Tree, Adaptive Neuro-Fuzzy Inference System, and Mahalanobis Distance Support Vector Machine. The authors utilized a relatively new dataset called UNSW-NB15 that contained features collected from HTTP and MQTT. Their proposed method revealed results superior to conventional methods. Another optimization method used for NIDSs to boost ML models' performance is the Moth–Flame Optimizer (MFO) algorithm [46]. The authors used Three NIDS datasets (KDDCUPP9, NSL-KDD, and UNSW-NB15) for performance evaluation. The MFO-based NIDS model was able to achieve classification accuracy greater than 89% for these datasets, which can be considered an outstanding result.

The authors of [47] hybridized the Harris Hawks Optimization (HHO) with the Particle Swarm Optimization (PSO) to find the best parameter values for their SVM-based IDS model. The proposed method was tested and evaluated using the NSL-KDD dataset. The obtained results proved the robust performance of the proposed method in finding the best parameter values for the Support Vector Machine, where it outperformed all compared methods, including C4.5 and K-nearest neighbor. In another study, optimization improvement utilizing the Salp Swarm Optimization and the chaotic map was proposed along with the LightGBM classifier to address the feature selection problem and produce an effective intrusion detection system for MQTT-IoT networks [48]. MQTTset, MQTT-IoT-IDS2020, and MC-IoT datasets were utilized to evaluate the performance of the proposed method. The results showed efficient performance in achieving the best outcomes.

The Differential Evolution (DE) algorithm was adapted by [49] to find the best features and accuracy to enhance and reduce the IDS processing time. These features were tested and evaluated using the extreme learning machine classifier. The NSL-KDD dataset was utilized to examine the adapted algorithm. The results showed good performance by the adapted algorithm compared to other methods. Kumar et al. [50] combined the parameters of the Hunger Games Search algorithm with the Remora Optimization Algorithm to find the best features for the IDS using a real-world Aegean Wi-Fi intrusion dataset. The achieved outcomes by the proposed method proved its high performance in obtaining the best results compared to the other methods.

The Butterfly Optimization Algorithm was adapted and hybridized with the Black Widow Optimization algorithm to enhance the local search capabilities of the Butterfly Optimization Algorithm and handle the feature selection problem of network IDS efficiently [51]. The UNSW-NB15 dataset was used in this study to validate the performance of the proposed hybrid method. The experimental results presented the high searching mechanism of the proposed method in finding the best solutions. Another hybrid method was proposed by [52] to address intrusion data classification problems and enhance detection accuracy in IoT environments. The UNSW-NB 15 and NSL-KDD datasets were utilized to evaluate the proposed method in finding the optimal solutions. The Random Forest classifier was also used to classify the attacks. The proposed method outperformed other methods reviewed in their study.

Many optimization methods were proposed in the literature to solve the feature selection problem for the IDSs over different datasets. The authors of [53] utilized deep learning and metaheuristic optimization algorithms to create effective feature extraction and selection techniques. Specifically, they used a one-dimensional Convolutional Neural Network (CNN) for feature extraction and the Reptile Search Algorithm (RSA) to select an optimal feature subset to reduce data dimensionality and improve classification accuracy. To evaluate the performance of their proposed techniques, the authors used various well-known datasets, such as KDDCup-99, NSL-KDD, CICIDS-2017, and BoT-IoT. Aziz and Alfoudi [54] adopted and enhanced Particle Swarm Optimization to improve its convergence to find the best number of features with optimal accuracy using NSL-KDD dataset. The genetic algorithm was adapted to address the same problem and dataset [55]. The authors proposed a new model based on logistic regression and a decision tree for further solution enhancements. Imran et al. also utilized the same dataset to evaluate the Cuckoo Search Optimization adapted to optimize the artificial neural network parameters and find better feature selection solutions [56]. Also, Alweshah et al. employed the shuffled shepherd optimization to solve the feature selection problem using the K-nearest neighbor classifier [57]. This study evaluated the utilized optimization method based on nine well-known IoT datasets. In [58], the Sea Lion Optimization Algorithm and Rider Optimization Algorithm were combined to enhance the optimization behavior in tuning the ensemble classifier. the authors of [59] adapted the Whale Optimization Algorithm to detect network attacks. In another study, the authors of [60] hybridized the searching behavior of the Artificial Gorilla Troops Optimizer (AGTO) with the Bird Swarm Algorithm to the AGTO searching capabilities and improved its feature selection solutions using UNSW-NB15, NSL-KDD, BoT-IoT, and CICIDS-2017 datasets. Also, [61] improved the searching capabilities of the Grey Wolf Optimizer by combining its parameters with the Particle Swarm Optimizer to find the optimal solution for the same problem utilizing NSL-KDD dataset. K-means and SVM algorithms were used to measure the performance of the solutions. Table 1 summarizes these manuscripts highlighting their purpose and utilized methods, classifiers, and datasets.

Based on the above summary of previous works established to design NIDSs using ML with some efficient optimization methods such as swarm intelligence and evolutionary algorithms, we believe that this area of research is still very active due to the nature of features that vary from one dataset to another. Therefore, the performance of any ML with an optimization method is not guaranteed, as emphasized in the "No Free Lunch" theorem of optimization [17], which states that there is no single superior optimization method for all kinds of optimization problems or even for a single optimization problem with different instances like the feature selection problem in NIDS. Therefore, there is still a window for utilizing other metaheuristic algorithms, such as the Arithmetic Optimization Algorithm (AOA), to boost the ML process for NIDS.

In this work, we utilized a modified version of the Arithmetic Optimization Algorithm (AOA) to propose an intrusion detection system that relies on a few NetFlow features to detect attacks. Minimizing the number of utilized features translates into a faster prediction time and lower storage space, which is crucial for IoT devices with limited resources. The proposed NIDS was tested over four complete datasets (released in 2021) with millions of network data flows and achieved an accuracy of up to 99% and 98% in binary and multi-classification, respectively. Our research significance can be outlined by the reduced feature dimensionality using an optimization algorithm and the high detection accuracy, which was rigidly tested over four different and recent datasets.

**Table 1**
A summary of optimization use in IDSs.

| NO. | Study | Method | Classifier | Dataset |
|---|---|---|---|---|
| 1 | Enhanced Coyote Optimization with LSTM Based Cloud-Intrusion Detection [41] | Enhanced Coyote Optimization with Deep Learning | long short term memory (LSTM) | NSL-KDD |
| 2 | Network Intrusion Detection Based on Feature Selection and Hybrid Metaheuristic Optimization [42] | Hybridized grey wolf optimizer with dipper throated optimizer | | IoT-IDS |
| 3 | Hybrid Grey Wolf and Dipper Throated Optimization in Network Intrusion Detection Systems [43] | Hybridized grey wolf optimizer with dipper throated optimizer | KNNs | RPL-NIDS17 |
| 4 | Voting Classifier and Metaheuristic Optimization for Network Intrusion Detection [44] | Hybridized whale optimization algorithm with dipper throated optimizer | Voting | |
| 5 | Improved Ant Colony Optimization and Machine Learning Based Ensemble Intrusion Detection Model [45] | Improved Ant Colony Optimization | Distance Decision Tree, Adaptive Neuro-Fuzzy Inference System and Mahalanobis Distance Support Vector Machine | UNSW-NB15 |
| 6 | A new intrusion detection system based on Moth–Flame Optimizer algorithm [46] | Moth–Flame Optimization | Decision Tree | KDDCUPP9, NSL-KDD, and UNSW-NB15 |
| 7 | An intelligent intrusion detection system for distributed denial of service attacks: A support vector machine with hybrid optimization algorithm based approach [47] | Hybrid Harris Hawks optimization with particle swarm optimization | support vector machine | NSL-KDD |
| 8 | An efficient intrusion detection system for MQTT-IoT using enhanced chaotic salp swarm algorithm and LightGBM [48] | chaotic salp swarm algorithm | LightGBM | MC-IoT, MQTT-IoT-IDS2020, and MQTTset |
| 9 | Wrapper feature selection method based differential evolution and extreme learning machine for intrusion detection system [49] | differential evolution | extreme learning machine | NSL-KDD |
| 10 | An intellectual intrusion detection system using Hybrid Hunger Games Search and Remora Optimization Algorithm for IoT wireless networks [50] | Hybrid Hunger Games Search and Remora Optimization Algorithm | support vector machine | AWID |
| 11 | Application of Improved Butterfly Optimization Algorithm Combined with Black Widow Optimization in Feature Selection of Network Intrusion Detection [51] | Hybrid Butterfly Optimization Algorithm with Black Widow Optimization | | UNSW-NB15 |
| 12 | A Hybrid Spider Monkey and Hierarchical Particle Swarm Optimization Approach for Intrusion Detection on Internet of Things [52] | Hybrid Spider Monkey with Hierarchical Particle Swarm Optimization | Random Forest | NSL-KDD and UNSW-NB 15 |
| 13 | Feature Selection of The Anomaly Network Intrusion Detection Based on Restoration Particle Swarm Optimization [54] | Restoration Particle Swarms Optimization | Random Forest | NSL-KDD |
| 14 | Intrusion Detection System for IoT Based on Deep Learning and Modified Reptile Search Algorithm [53] | Reptile search algorithm | KNN | KDDCup-99, NSL-KDD, CICIDS-2017, and BoT-IoT |
| 15 | Intrusion detection system using hybrid classifiers with meta-heuristic algorithms for the optimization and feature selection by genetic algorithm [55] | genetic algorithm | logistic regression and decision tree | NSL-KDD |
| 16 | Intrusion detection in networks using cuckoo search optimization [56] | cuckoo search optimization | | NSL-KDD |
| 17 | Intrusion detection using optimized ensemble classification in fog computing paradigm [58] | Rider Sea Lion Optimization | ensemble | |
| 18 | Intrusion detection for IoT based on a hybrid shuffled shepherd optimization algorithm [57] | hybrid shuffled shepherd optimization algorithm | KNN | |
| 19 | Intrusion Detection Network Attacks Based on Whale Optimization Algorithm [59] | Whale Optimization Algorithm | | |
| 20 | An Effective Feature Selection Model Using Hybrid Metaheuristic Algorithms for IoT Intrusion Detection [60] | Hybrid Gorilla Troops Optimizer with the bird swarms algorithm | KNN | UNSW-NB15, NSL-KDD, BoT-IoT, and CICIDS-2017 |
| 21 | An enhanced Grey Wolf Optimizer based Particle Swarm Optimizer for intrusion detection system in wireless sensor networks [61] | Hybrid Grey Wolf Optimizer with Particle Swarm Optimizer | k-mean and support vector machine | NSL-KDD |

## 3. Methodology

This section presents the architecture of the research methodology, starting with an illustration of the proposed system's architecture. As shown in Fig. 1, the proposed system consists of three main phases: (1) Data Preprocessing, (2) Features Selection, and (3) Classification Using Machine Learning.

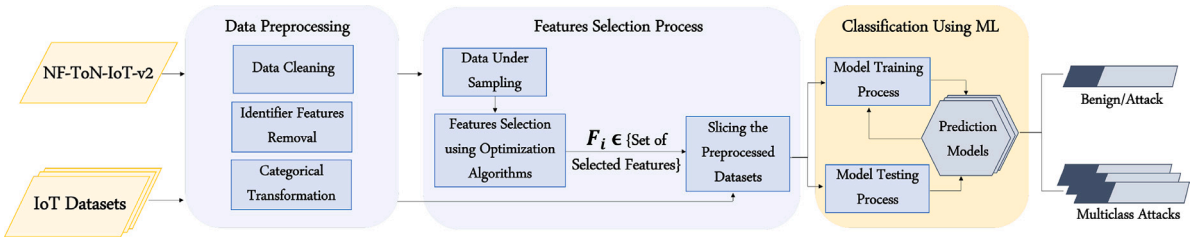



**Fig. 1.** The proposed system architecture.

### 3.1. Data preprocessing

This subsection describes the steps within the preprocessing phase. The purpose of this phase is to prepare the data for analysis.

#### 3.1.1. Data cleaning

The cleaning process involves removing incomplete, incorrect, duplicate, or irrelevant rows and columns. The goal of data cleaning is to improve the data quality, which will, in turn, enhance the performance and efficiency of the analysis process. The data cleaning process consists of row and column cleaning. In the row-cleaning process, all records that were either incomplete or contained noise (e.g., missing, null values, or duplicates) were dropped, which resulted in the removal of 131 rows. Any unnamed columns were removed from the dataset during the column-cleaning process. Additionally, for the multi-classification experiments, the Label feature was dropped since it is highly correlated with the Attack feature. On the other hand, the Attack feature was removed for the bi-classification experiments. Finally, features that only had a single value, such as FTP_COMMAND_RET_CODE, were also removed as they did not contribute to the classification process.

The cleaned NF-ToN-IoT-v2 dataset consists of complete rows with no missing values or duplicates.

#### 3.1.2. Identifying features removal

The following four identifying features were dropped to avoid any bias during the classification process: IPV4_SRC_ADDR, IPV4_DST_ADDR, L4_SRC_PORT, and L4_DST_PORT. These features are related to the IP addresses and port numbers of the source and destination.

#### 3.1.3. Categorical transformation

The categorical transformation is vital and typical in the preprocessing stage. It enables the classifier to learn better from the fed dataset. Categorical transformation is obligatory for any classifier model since it can only receive numeric values. In NF-ToN-IoT-v2 dataset, the attack feature contains ten classes (one benign class and nine attack classes) which were encoded into numbers from 0 to 9. On the other hand, the binary label feature is encoded into 0 for the no-attack (benign) and 1 for the attack (anomaly) class.

### 3.2. Features selection process using AOA

The section below presents the feature selection process, including the data under sampling process and a presentation of the optimization algorithm used to select the best features pattern.

As shown in Fig. 1, the NF-ToN-IoT-v2 dataset is used to select the best features that represent each attack class discriminantly. Once the optimizer provides the best attack detection features, they are applied to slice any preprocessed NetFlow-featured dataset. Afterward, the sliced preprocessed datasets are fed into the machine learning models to determine the accuracy of the optimizer-based selected features and validate those features as a standard feature set to be used to detect any attack pattern on any NetFlow-based dataset.

#### 3.2.1. Data under sampling process

The large dataset size (around 17M records) makes its use with the optimization algorithm computationally greedy. Thus, the undersampling process was applied while maintaining the proportions of each attack class in the dataset.

Stratified Random Sampling method [62] is used to randomly undersample the dataset, where the records belonging to the minority attack classes are kept, and the records belonging to the majority attack classes are decreased. In our case, all attack classes with less than 20k records were considered a minority. Otherwise, they are the majority and reduced randomly, as shown in Fig. 2.

As can be seen in Fig. 2, the samples of the attack classes Backdoor (16,809), MITM (7723), and Ransomware (3425) were kept. However, the samples from the other classes were reduced. The Benign, DDoS, DoS, Injection, Password, Scanning, and XSS samples were reduced from 6.09 M, 2.02 M, 0.7 M, 0.68 M, 1.15 M, 3.78 M, and 2.45 M to 350 K, 116 K, 40 K, 39 K, 66 K, 217 K, and 141 K, respectively. In total, 1 M randomly selected samples were fed to the AOA.

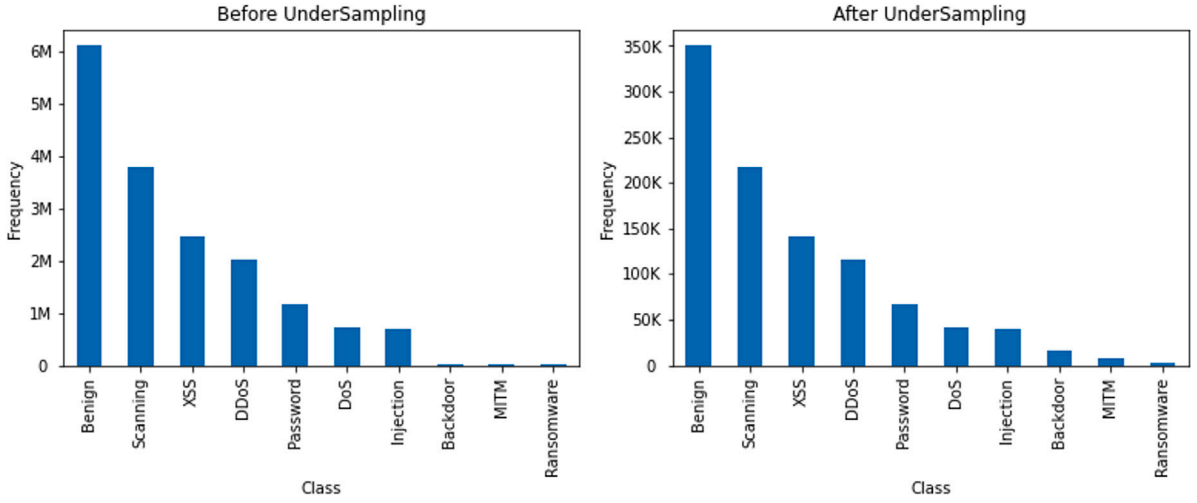



**Fig. 2.** Straight random under sampling.

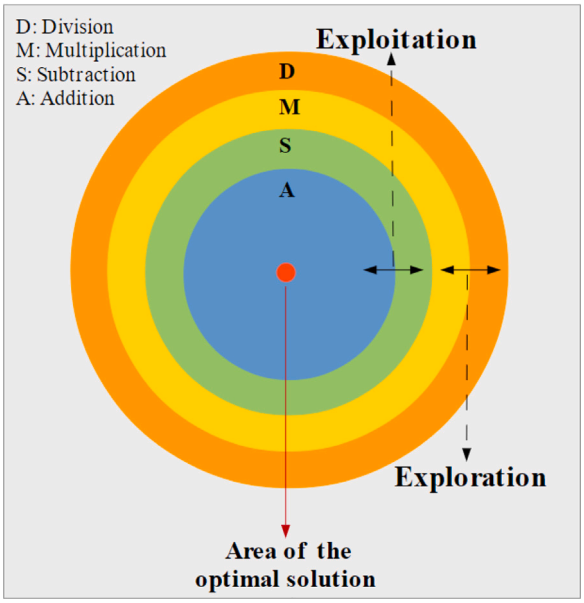



**Fig. 3.** Arithmetic operators' hierarchy (top-down dominance declines).

### *3.2.2. AOA-based intrusion detection*

The Arithmetic Optimization Algorithm (AOA) is a population-based metaheuristic search technique proposed by Abualigah et al. [18]. Mainly, AOA is inspired by the four basic arithmetic operations in math: Multiplication ($M$), Division ($D$), Subtraction ($S$), and Addition ($A$). Similar to any population-based algorithm, AOA's optimization process includes two main phases: exploration and exploitation. Due to these arithmetic operations behavior, the $M$ and $D$ are applied in the exploration phase, generating high-distributed values in the search space. Whereas $A$ and $S$ are utilized in the exploitation phase to produce high-dense results in the search space. Fig. 3 depicts the sequence of the four arithmetic operations and their superiority from outside to inside. Below, we present the mathematical models of the AOA.

**Step 1: Initialization phase** As a population-based optimization algorithm, AOA starts with a random generalization of a set of candidate solutions ($X$), as illustrated in Eq. (1). In each iteration, the fittest candidate solution is considered the best-so-far

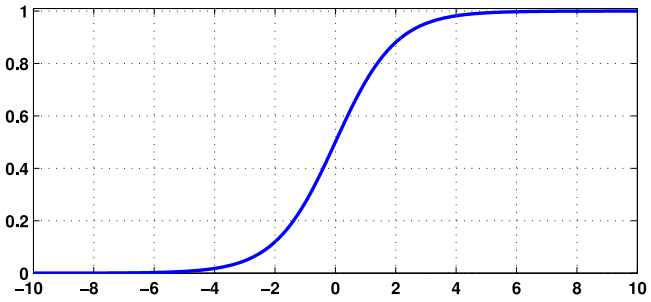



**Fig. 4.** S-shape function.

solution.

$$X = \begin{bmatrix} x_{1,1} & \cdots & x_{1,j} & x_{1,n-1} & x_{1,n} \\ x_{2,1} & \cdots & x_{2,j} & \cdots & x_{2,n} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ x_{N-1,1} & \cdots & x_{N-1,j} & \cdots & x_{N-1,n} \\ x_{N,1} & \cdots & x_{N,j} & x_{N,n-1} & x_{N,n} \end{bmatrix} \tag{1}$$

AOA utilizes a dynamic function called Math Optimizer Accelerated ($MOA$) to determine the search phase: exploration or exploitation. MOA can be calculated using Eq. (2)

$$MOA(C_{Iter}) = Min + C_{Iter} \times (\frac{Max - Min}{C_{Iter}}) \tag{2}$$

in which the maximum iteration is denoted by $M_{Iter}$ while $C_{Iter}$ is the current iteration ranging between 1 and $M_{Iter}$. $Max$ and $Min$ represent the maximum and minimum values of the $MOA$ function.

**Step 2: Exploration phase** As mentioned earlier, the $D$ and $M$ are the operations applied in the exploration phase. The exploration phase is entered if the accelerated function ($MOA$) value is less than $r_1$, where $r_1$ is a uniformly distributed random number between [0, 1]. As the exploration operators ($M$ and $D$) have high dispersion, the search space could be explored over diverse areas. The exploration phase is given in Eq. (3). As shown in Eq. (3), the Division ($D$) search is performed when the random number ($r_2$) is less than 0.5; otherwise, the Multiplication ($M$) search will be executed.

$$x_{i,j}(C_{Iter}+1) = \begin{cases} best(x_j) \div (MOP + \varepsilon) \times ((UB_j - LB_j) \times \mu + LB_j), & r_1 < 0.5 \\ best(x_j) \times MOP \times ((UB_j - LB_j) \times \mu + LB_j), & \text{otherwise} \end{cases} \tag{3}$$

Referring to the equation notations, $x_i(C_{Iter}+1)$ is the next iteration of the $i$th solution, $x_{i,j}(C_{Iter})$ represents the $j$th position of $i$th solution in the current iteration, and the $j$th position of the best-so-far solution is denoted by $best(x_j)$. The upper bound and the lower bound values of the $j$th solution are represented, respectively, by $UB_j$ and $LB_j$. $\epsilon$ denotes a small integer number, and $\mu$ is a control parameter of the search process that is set to 0.5 as proposed by Abualigah et al. [18]. Finally, Math Optimizer Probability ($MOP$) is a non-linearly decreasing coefficient from 1 to 0 with each iteration and can be expressed mathematically using Eq. (4)

$$MOP(C_{Iter}) = 1 - \frac{C_{Iter}^{\frac{1}{\alpha}}}{M_{Iter}^{\frac{1}{\alpha}}} \tag{4}$$

where the maximum iteration and the current iteration are denoted by $M_{Iter}$ and $C_{Iter}$, respectively. $\alpha$ is a sensitive parameter that reflects the efficiency of exploitation across iterations; Abualigah et al. [18] suggested fixing its value to 5.

**Step 3: Exploitation phase** The Subtraction ($S$) and Addition ($A$) operators are utilized in the exploitation stage. This phase is executed when the accelerated function ($MOA$) value (see Eq. (2)) is greater than or equal to $r_1$. In contrast to the exploration operators, $S$ and $A$ have low dispersion, which allows them to get closer to the best solution. Similar to the exploration, the $S$ search is performed when the random number ($r_3$) < 0.5, otherwise, the $A$ search will be executed. Eq. (5) shows the mathematical model of the exploitation stage.

$$x_{i,j}(C_{Iter}+1) = \begin{cases} best(x_j) - MOP \times ((UB_j - LB_j) \times \mu + LB_j), & r_3 < 0.5 \\ best(x_j) + MOP \times ((UB_j - LB_j) \times \mu + LB_j), & \text{otherwise} \end{cases} \tag{5}$$

To recapitulate, the optimization process of AOA starts with the random generation of the populations. After that, if the value of the accelerated function ($MOA$) is less than the random number ($r_1$), the exploration search is performed (i.e., $M$ and $D$ operators).

Otherwise, the exploitation search is conducted (i.e., $S$ and $A$ operators). The AOA algorithm is eventually terminated when the end condition is satisfied ($M_{Iter} > C_{Iter}$). Algorithm 1 shows the pseudo-code of AOA.

**Algorithm 1** Arithmetic Optimization Algorithm

**Step 1: Initialization Phase**
  Initialize the algorithm's parameters $\alpha, \mu$
  Initialize position of the solution populations $i = 1, 2, ..., N$
**while** $C_{Iter} \leq M_{Iter}$ **do**
  Compute fitness function
  Find $x_{best}$
  Apply Eq. (2) to update $MOA$
  Apply Eq. (3) to update $MOP$
  **for** ($i = 1$ to $N$) **do**
    **for** ($j = 1$ to $n$) **do**
      Generate random values for $r_1$,$r_2$, and $r_3$ between [0, 1]
      **if** $r_1 \geq MOA$ **then**
**Step 2: Exploration Phase**
        **if** $r_2 \geq 0.5$ **then**
          Apply $D$ operator
          Update new position of $i^{th}$ solution using $1^{st}$ rule of $Eq.(3)$
        **else**
          Apply $M$ operator
          Update new position of $i^{th}$ solution using $2^{nd}$ rule of $Eq.(3)$
      **else**
**Step 3: Exploitation Phase**
        **if** $r_3 \geq 0.5$ **then**
          Apply $S$ operator
          Update new position of $i^{th}$ solution using $1^{st}$ rule of $Eq.(5)$
        **else**
          Apply $M$ operator
          Update new position of $i^{th}$ solution using $2^{nd}$ rule of $Eq.(5)$
  $C_{Iter} = C_{Iter} + 1$
**return** $x_{best}$

### 3.2.3. Transfer function

In order to change the continuous values into the binary domain to deal with the FS problem of NIDS, the sigmoid (S-shaped) transfer function is utilized in the optimization framework of the AOA method. In the solution, the value of 1 means the corresponding feature is selected. On the other hand, the value of 0 indicates that the feature is not selected. The used S-shaped transform function formulated in Eq. (6).

$$S\left(x_{i,j}(t)\right) = \frac{1}{1 + e^{-2(x_{i,j}(t))}} \quad \forall j = (1, 2, \ldots, m) \tag{6}$$

where $x_{i,j}(t)$ is the value $i$ in the solution $j$ at iteration $t$ (see Fig. 4).

Then, each decision variable in solution $j$ is assigned by a binary value with regard to its transfer vector $S\left(x_{i,j}(t)\right)$ using Eq. (7).

$$x_{i,j}(t) = \begin{cases} 1 & rnd < S\left(x_{i,j}(t)\right) \\ 0 & Otherwise \end{cases} \tag{7}$$

where $rnd$ is a randomly generated value between 0 and 1 (i.e., $rnd \in [0, 1]$).

### 3.2.4. Objective function

The Objective function, which tries to find the best trade-off between a minimum number of features and the highest classification rate, is utilized to evaluate the FS solution. The mathematical formulation of the objective function is presented in Eq. (8).

$$fit = \alpha \times \gamma + \beta \frac{|R|}{|N|} \tag{8}$$

Note that the classification error rate is represented by $\gamma$. $|R|$ and $|N|$ are the number of features selected and the total number of features, respectively. $\frac{|R|}{|N|}$ is the ratio of selecting the features. The two parameters $\alpha$ and $\beta$ are used to find the suitable balance between the importance of the number of features and the classification accuracy (i.e., $\alpha$ and $\beta \in [0, 1]$ and $\alpha + \beta = 1$).

## 3.3. Classification using machine learning

Several widely used machine learning algorithms were applied to construct a model that can accurately differentiate between the different attack classes of each NF dataset. The primary purpose is to test the proposed features and establish a new set of NF features that would serve as a standard to predict attacks.

The algorithms were selected from machine learning theories, such as tree-based algorithms and Naïve Bayes theorem-based algorithms. Tree-based algorithms are widely used for classification problems by constructing a tree-like structure to make predictions. On the other hand, Naïve Bayes theorem-based algorithms classification techniques are based on Bayes' Theorem with a hypothesis of independence between predictors. As experienced, the four tree-based algorithms were better in predicting attacks for the four NetFlow datasets.

The first algorithm used to detect anomalies is the Decision Tree (DT) [27]. It constructs a decision tree model based on training instances from the dataset to test its capability in correctly classifying new testing dataset instances. The decision tree model is easy to train since it is a single model based on a set of if-then rules to predict new instances. However, it is not flexible and can overfit the training dataset. Several methods are used to build the decision tree, such as ID3 and C4.5, and Classification and Regression Trees (CART) [63]. This research will use the CART algorithm provided by default in Scikit learn Python library.

The second used algorithm in our IDS is Random Forest (RF) [26]. As the name indicates, it consists of an ensemble of decision trees that form a forest. It has been utilized in various IDSs and research papers [64–66]. The model is created using bagging (bootstrap aggregation) techniques. The algorithm's performance is measured by the majority voting or the average of the trees' performance. The RF tends to find the globally optimal solution rather than the local optimum, as in decision trees. As a result, RF overcomes the limitations of the decision tree algorithm. It reduces the overfitting to the datasets and outperforms the decision tree performance [67], but it is also more computationally demanding since it requires training many decision tree models.

The third utilized algorithm in our IDS is Extra Trees (ET), also called Extremely Randomized Trees [9]. Similar to RF, it builds a set of decision trees during the training phase. The main difference between ET and RF is that, during the training phase, the RF does not use the entire dataset to build the model but uses bootstrapped subsets of the dataset. In contrast, ET uses all the instances of the training dataset. Additionally, the RF selects the best node to split the tree at, whereas ET selects it randomly. These differences translate into a lower bias due to using all samples and not a bootstrap replica and reducing the variance due to choosing the split node randomly. In terms of computational time, the ET algorithm is faster since there is no need to look for the best-split node, which consumes a lot of time. In terms of performance, the ET can be better than RF (or similar) depending on the dataset quality characteristics, such as features' relevancy and noisiness.

The fourth algorithm used in our IDS is Extreme Gradient Boosting (XGB) [28], an improved implementation of the gradient boosting ensemble algorithm. As in RF and ET, the XGB is an enhancement to decision trees. XGB aims to enhance classification performance by growing trees sequentially in contrast to the parallel approach in RF and ET. The main concept of XGB is to allow trees to learn from the errors generated by previous trees by stacking them sequentially. This approach enables us to solve one of the RF drawbacks by reducing the bias and improving the predictive accuracy. However, XGB is computationally heavy due to the extensive grid search used to determine the best hyperparameters of XGB models.

The final algorithm used in this research is Naïve Bayes (NB) [25]. It is a probabilistic supervised machine learning algorithm used in classification tasks. It assumes that the presence of the features in a class is entirely independent. The Naïve Bayes model is simple to build and is practical for massive datasets [68].

These supervised ML algorithms are used for binary classification (benign vs. attack classes) and Multi-classification (between all classes, including benign and the various attack classes).

## 4. Experiments and results

This section describes the conducted experiments and provides detailed results and analysis.

### 4.1. Experiment design

Several optimization models used in the feature selection phase were trained and tested using the preprocessed, sliced, and undersampled NF-ToN-IoT-v2 dataset. After that, 80% of the dataset were selected for training and 20% for testing. To find the best model with the best performance, 50 iterations and 30 independent runs for 30 candidate solutions were executed using Decision Tree as an objective function. The optimization models were implemented on Matlab and trained on an Intel Core i9-7960X X-Series (2.8 GHz 16-Core) server.

WSO, GWO, BAT, and AOA [18] optimization models were used in the feature selection process. The hyper-parameters of these algorithms were defined according to the state of art implementation.

The best set of features that independently represented different classes of each dataset, provided by the best optimization model, was used to filter the four datasets used in this research work. The classification machine learning models DT, RF, ET, XGB, and NB were implemented with the default hyper-parameters using Scikit-learn Python library and trained using Intel Core i9-7960X X-Series 2.8 GHz 16-Core.

### 4.2. Datasets description

Our work utilizes four updated versions of existing NIDS datasets that have been standardized into a NetFlow format with 43 common features by Sarhan et al. [9]. The common features between the four datasets are shown in Table 2 [69]. Below, we provide further details on each dataset:

**NF-CSE-CIC-IDS2018-v2** is a NetFlow-based dataset that contains 18,893,708 flows distributed between 16,635,567 benign records and 2,258,141 attack records. This dataset includes six different attack scenarios: Bruteforce, Bot, Denial of Service (DoS),

**Table 2**
NetFlow common 43 features [69].

| Feature | Description |
|---|---|
| 1. IPV4_SRC_ADDR | IPv4 source address |
| 2. IPV4_DST_ADDR | IPv4 destination address |
| 3. L4_SRC_PORT | IPv4 source port number |
| 4. L4_DST_PORT | IPv4 destination port number |
| 5. PROTOCOL | IP protocol identifier byte |
| 6. L7_PROTO | Layer 7 protocol (numeric) |
| 7. IN_BYTES | Incoming number of bytes |
| 8. IN_PKTS | Incoming number of packets |
| 9. OUT_BYTES | Outgoing number of bytes |
| 10. OUT_PKTS | Outgoing number of packets |
| 11. TCP_FLAGS | Cumulative of all TCP flags |
| 12. CLIENT_TCP_FLAGS | Cumulative of all client TCP flags |
| 13. SERVER_TCP_FLAGS | Cumulative of all server TCP flags |
| 14. FLOW_DURATION_MILLISECONDS | Flow duration in milliseconds |
| 15. DURATION_IN Client | to Server stream duration (msec) |
| 16. DURATION_OUT | Client to Server stream duration (msec) |
| 17. MIN_TTL | Min flow TTL |
| 18. MAX_TTL | Max flow TTL |
| 19. LONGEST_FLOW_PKT | Longest packet (bytes) of the flow |
| 20. SHORTEST_FLOW_PKT | Shortest packet (bytes) of the flow |
| 21. MIN_IP_PKT_LEN | Len of the smallest flow IP packet observed |
| 22. MAX_IP_PKT_LEN | Len of the largest flow IP packet observed |
| 23. SRC_TO_DST_SECOND_BYTES | Src to dst Bytes/sec |
| 24. DST_TO_SRC_SECOND_BYTES | Dst to src Bytes/sec |
| 25. RETRANSMITTED_IN_BYTES | Number of retransmitted TCP flow bytes (src → dst) |
| 26. RETRANSMITTED_IN_PKTS | Number of retransmitted TCP flow packets (src → dst) |
| 27. RETRANSMITTED_OUT_BYTES | Number of retransmitted TCP flow bytes (dst → src) |
| 28. RETRANSMITTED_OUT_PKTS | Number of retransmitted TCP flow packets (dst → src) |
| 29. SRC_TO_DST_AVG_THROUGHPUT | Src to dst average thpt (bps) |
| 30. DST_TO_SRC_AVG_THROUGHPUT | Dst to src average thpt (bps) |
| 31. NUM_PKTS_UP_TO_128_BYTES | Packets whose IP size <= 128 |
| 32. NUM_PKTS_128_TO_256_BYTES | Packets whose IP size > 128 and <= 256 |
| 33. NUM_PKTS_256_TO_512_BYTES | Packets whose IP size > 256 and <= 512 |
| 34. NUM_PKTS_512_TO_1024_BYTES | Packets whose IP size > 512 and <= 1024 |
| 35. NUM_PKTS_1024_TO_1514_BYTES | Packets whose IP size > 1024 and <= 1514 |
| 36. TCP_WIN_MAX_IN | Max TCP Window (src → dst) |
| 37. TCP_WIN_MAX_OUT | Max TCP Window (dst → src) |
| 38. ICMP_TYPE | ICMP Type * 256 + ICMP code |
| 39. ICMP_IPV4_TYPE | ICMP Type |
| 40. DNS_QUERY_ID | DNS query transaction Id |
| 41. DNS_QUERY_TYPE | DNS query type (e.g. 1=A, 2=NS..) |
| 42. DNS_TTL_ANSWER | TTL of the first A record (if any) |
| 43. FTP_COMMAND_RET_CODE | FTP client command return code |

Distributed Denial of Service (DDoS), Infiltration, and Web Attacks. Some of these types are subcategorized. For example, Bruteforce attacks can be either FTP-BruteForce or SSH-Bruteforce. The Web attacks also comprise Brute Force-Web, Brute Force-XSS, and SQL Injection. Moreover, DoS contains GoldenEye, Hulk, and Slowloris, whereas the DDoS attack includes HOIC, LOIC-UDP, and LOIC-HTTP.

**NF-ToN-IoT-v2** is an IoT traffic dataset consisting of a total of 16,940,496 data rows. The total attack records are 10,841,027, while the benign records are 6,099,469. The attack samples contain nine different types: Backdoor, DoS, DDoS, Injection, Man in the Middle (MITM), Password, Ransomware, Scanning, and Cross-site Scripting (XSS).

**NF-UNSW-NB15-v2** contains network data with NetFlow-formatted features. It has a total data flows of 2,390,275, of which 2,295,222 are benign, and 95,053 are attack samples. The anomaly class contains nine attack types: Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, and Worms.

**NF-BoT-IoT-v2** is an IoT network data that includes 37,763,497 samples. Most of these samples are attacks (37,628,460), with 135,037 benign flows. The attack can be either DDoS, DoS, Reconnaissance, or Theft.

Table 3 below summarizes the distribution of the classes for each dataset [9].

### *4.3. Evaluation measures*

To evaluate the results of our experiments, we used the following evaluation measures: Precision, Detection Rate, F1-score, False Acceptance rate, Accuracy, and Dimensionality Reduction rate. These metrics were used to measure the correctness and performance of the proposed IDS system in detecting the benign vs. attack classes in binary classification. In the multi-classification process, the Precision, Detection Rate, F1-score, False Acceptance rate, and Accuracy metrics [70] were calculated for each class. Fig. 5 shows

**Table 3**
Breakdown of the datasets classes [9].

| NF-CSE-CIC-IDS2018-v2 | | NF-ToN-IoT-v2 | | NF-UNSW-NB15-v2 | | NF-BoT-IoT-v2 | |
|---|---|---|---|---|---|---|---|
| Class | Count | Class | Count | Class | Count | Class | Count |
| Benign | 16,635,567 | Benign | 6,099,469 | Benign | 2,295,222 | Benign | 135,037 |
| Bot | 143,097 | Backdoor | 16,809 | Analysis | 2,299 | DDoS | 18,331,847 |
| Brute Force -Web | 2,143 | DDoS | 2,026,234 | Backdoor | 2,169 | DoS | 16,673,183 |
| Brute Force -XSS | 927 | DoS | 712,609 | DoS | 5,794 | Reconnaissance | 2,620,999 |
| DDOS attack-HOIC | 1,080,858 | Injection | 684,465 | Exploits | 31,551 | Theft | 2,431 |
| DDOS attack-LOIC-UDP | 2,112 | MITM | 7,723 | Fuzzers | 22,310 | | |
| DDoS attacks-LOIC-HTTP | 307,300 | Password | 1,153,323 | Generic | 16,560 | | |
| DoS attacks-GoldenEye | 27,723 | Ransomware | 3,425 | Reconnaissance | 12,779 | | |
| DoS attacks-Hulk | 432,648 | Scanning | 3,781,419 | Shellcode | 1,427 | | |
| DoS attacks-SlowHTTPTest | 14,116 | XSS | 2,455,020 | Worms | 164 | | |
| DoS attacks-Slowloris | 9,512 | | | | | | |
| FTP-BruteForce | 25,933 | | | | | | |
| Infilteration | 116,361 | | | | | | |
| SQL Injection | 432 | | | | | | |
| SSH-Bruteforce | 94,979 | | | | | | |
| **Total** | **18,893,708** | **Total** | **16,940,496** | **Total** | **2,390,275** | **Total** | **37,763,497** |

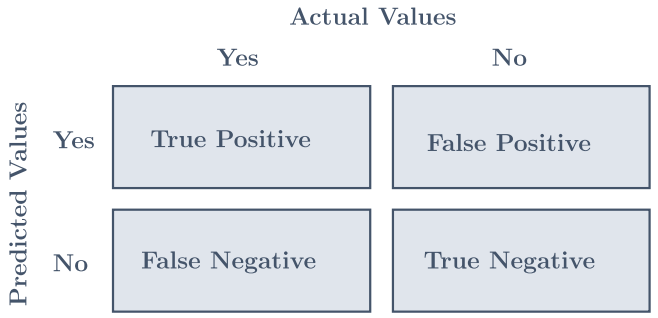



**Fig. 5.** The models' confusion matrix.

the confusion matrix of the classification, including True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN) results, which can be obtained for each class k, where $0 \leq k \leq n$.

As shown in Eq. (9), the precision measure is the ratio of actual attack records predicted successfully as an attack to the total number of records that were predicted as an attack.

$$Precision = \frac{TP}{(TP + FP)} \tag{9}$$

The detection rate measure (Eq. (10)) is the ratio of actual attack records predicted successfully as an attack to the total records in the attack class.

$$Detection\ Rate = \frac{TP}{(TP + FN)} \tag{10}$$

$F1 - Score$ is defined, in Eq. (11), as the harmonic mean of Precision and Detection Rate measures.

$$F1 - Score = \frac{2 * (detection\ rate * Precision)}{(detection\ rate + Precision)} \tag{11}$$

The False Acceptance rate (FAR), also called the False Positive rate (FPR), represents the proportion of Benign records incorrectly identified by the system as Attacks records. As presented in Eq. (12), it is the ratio of the number of Benign records classified as Attacks records to the total number of Benign records.

$$False\ Acceptance\ rate = \frac{(FP)}{(TN + FP)} \tag{12}$$

The Accuracy measure is, as presented in Eq. (13), the ratio of all correct detection records classes (Attack and Benign) to the total number of detection records classes (TP + FP + TN + FN).

$$Accuracy = \frac{(TP + TN)}{(TP + FP + TN + FN)} \tag{13}$$

The Dimensionality Reduction rate (Eq. (14)) is the ratio of selected features to the total number of features in each dataset.

$$Dimensionality\ Reduction\ rate = \frac{number\ of\ selected\ features}{total\ number\ of\ features} \tag{14}$$

**Table 4**
Fitness function obtained by the methods.

| | BAOA | BBAT | BWSO | BGWO |
|---|---|---|---|---|
| AVG | 0.0302 | 0.0317 | 0.0315 | **0.0301** |
| STD | 3.81E−04 | **3.51E−04** | 4.90E−04 | **3.51E−04** |
| WORST | **0.0309** | 0.0325 | 0.0326 | 0.0314 |
| BEST | **0.0297** | 0.031 | 0.0306 | 0.0298 |

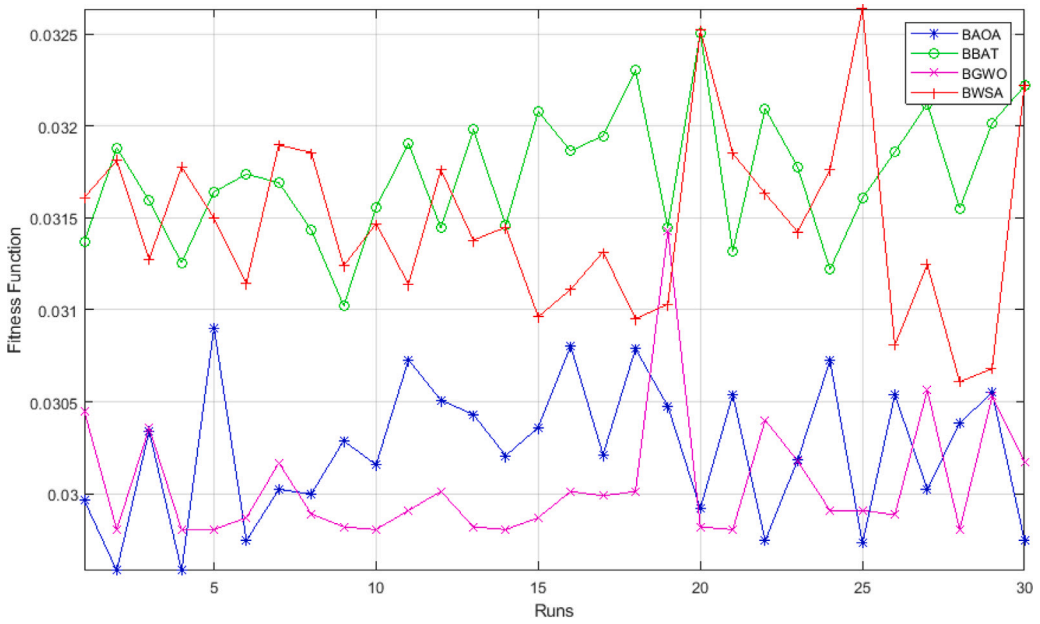



**Fig. 6.** Fitness function comparison between the methods.

**Table 5**
Evaluation results of the optimization algorithms based on accuracy.

| | BAOA | BBAT | BWSO | BGWO |
|---|---|---|---|---|
| AVG | 0.9717 | 0.9717 | **0.9718** | **0.9718** |
| STD | 4.33E−04 | 4.54E−04 | 3.29E−04 | **4.23E−04** |
| WORST | **0.9707** | 0.9705 | **0.9707** | 0.9701 |
| BEST | 0.9723 | **0.9724** | 0.9722 | 0.9722 |

### *4.4. Comparative evaluation*

As mentioned previously, four widely used optimization algorithms AOA, GWO, BAT, and WSO, were adopted and modified to deal with the binary nature of the feature selection problem and achieve the optimal error rate to identify the attacks and benign classes accurately. The newly proposed binary versions of AOA, GWO, BAT, and WSO were named BAOA, BGWO, BBAT, and BWSO, respectively. The performance of these optimization algorithms was evaluated per three primary measurements: error rate, achieved accuracy, and the number of selected features.

Table 4 presents the best, worst, average (AVG), and standard deviation (STD) error rates obtained by all utilized methods. The table shows that the BGWO achieved the best AVG error rate, whereas the BAOA achieved the second best by a low margin (0.0001). The BGWO and BBAT shared the same STD values with 3.51E−04, and the BAOA achieved the second-best STD with a low margin (0.3E−4). In terms of worst and best error rates, the BAOA attained the best results and solutions, where its worst and best solutions are the minimum among all compared methods, as shown in Fig. 6.

Similarly, the utilized methods show almost the same behavior in maximizing the accuracy of identifying the attacks. The BGWO and BWSO achieved the best AVG, and BAOA and BBAT were the second best, as shown in Table 5. Also, the BGWO and AOA attained the first and second best STD, respectively. Regarding the worst obtained solutions, the BAOA and BWSO achieved the maximum worst accuracy, whereas the BBAT earned the best-ever accuracy in comparison to the other methods.

Regarding the number of features obtained by each method, the BGWO attained the best AVG and STD with eight features and 5.87E−01, respectively. The BAOA came in second with 8.26 features and 1.31, respectively, as shown in Table 6. Also, the BGWO

**Table 6**
Evaluation results of the optimization algorithms based on number of selected features.

| | BAOA | BBAT | BWSO | BGWO |
|---|---|---|---|---|
| AVG | 8.2667 | 14.1 | 13.3333 | **8** |
| STD | 1.31E+00 | 2.02E+00 | 2.20E+00 | **5.87E−01** |
| WORST | 12 | 18 | 19 | **9** |
| BEST | **6** | 10 | 9 | 7 |

**Table 7**
Evaluation results of the optimization algorithms based on best solution.

| | BAOA | BBAT | BWSO | BGWO |
|---|---|---|---|---|
| Error Rate | 0.0297 | 0.031 | 0.0306 | 0.0298 |
| Accuracy | 0.9718 | 0.9719 | 0.9715 | 0.9720 |
| # Features | 7 | 12 | 9 | 8 |

obtained the minimum worst number of features, and the BAOA was the second. For the best solutions, the BAOA achieved the best solution with six features, whereas the BGWO with seven features.

Note that the results obtained by the BGWO and BAOA have fluctuated. In some cases, BGWO showed better results; in others, BAOA was better. Accordingly, to present and select the best method for addressing the feature selection problem using the NF-ToN-IoT-v2 dataset, the best solution with the best error rate was measured for all methods. The error rate was chosen in this comparison because it is the objective function of this study. Table 7 compares all methods. The table shows that the BAOA achieved the best error rate with the seven features. However, the BGWO obtained the best accuracy with a small margin (0.002).

Based on the accuracy and the number of features, we concluded that BAOA is the best optimizer. BAOA chose only seven features from the NF-ToN-IoT-v2 dataset to predict each class with 97.18% accuracy for multi-attacks classification. This means it is enough to select these seven features to detect the ten attacks and benign labels in the NF-ToN-IoT-v2 dataset. It is worth noting that while we only used seven out of the 43 features, our experiments utilized the entire network flows in each dataset (without sampling). The seven (7) selected features are: **'PROTOCOL', 'TCP_FLAGS', 'SERVER_TCP_FLAGS', 'MAX_IP_PKT_LEN', 'SRC_TO_DST_SECOND_BYTES', 'MAX_TTL', and 'TCP_WIN_MAX_IN'**.

### *4.5. Standard features for NetFlow datasets*

In this section, we describe our study to assess the validity of the features selected by the optimization algorithm. As stated earlier, our goal is to define a set of features that can be used to predict different attack types over any NetFlow network. For this, the 7 features selected from the **NF-ToN-IoT-v2** were utilized in models that operated over the other three NF datasets: **NF-CSE-CIC-IDS2018-v2**, **NF-UNSW-NB15-v2**, and **NF-BoT-IoT-v2**. The seven selected features were used to trim these NF datasets. Afterward, the datasets were fed to the five chosen machine learning algorithms: DT, RF, XGB, NB, and ET. For each machine learning model, 80% of the data was selected to train the model and 20% to test it. The ability to predict all classes (attacks and benign) was evaluated using five measurement metrics (PR, DR, F1, weighted avg, and Accuracy). A comparison between the five models across the four datasets is presented at the end of this section.

Tables 8 and 9 demonstrate a comparison of the validation results of multi-class and binary classification using the five machine-learning models with only seven features over NF-UNSW-NB15-v2 dataset. According to these two tables, we can observe that using seven features was enough to discriminate each class (out of 10 attack and benign classes) in the NF-UNSW-NB15-v2 dataset in the multi-classification (Table 8) and to differentiate between anomaly and begin instances in binary classification (Table 9) successfully. The classification of the analysis, backdoor, worms, and DoS attacks are still unreliable due to their small number of samples. The performance achieved by our models is very satisfactory in terms of ACC, PR, DR, and F1-scores using only seven features. In fact, our results are comparable to the results obtained by Sarhan et al. in [9] using 39 features with the ET model and Sayed in [37] using 37 features with the Convolutional Neural Networks (CNN) model.

Tables 10 and 11 demonstrate a comparison of the validation results of multi-class and binary classification using the five machine-learning models with only seven features over the NF-BoT-IoT-v2 dataset.

According to these two tables, we can also notice that using seven features was enough to predict each class (out of 5 attack and benign classes) in the NF-BoT-IoT-v2 dataset in the multi-classification (Table 10) and to differentiate between anomaly and begin instances in binary classification (Table 11) successfully. The detection of the Theft attack is relatively low due to the number of samples (2431 samples). However, the performance achieved by our models in terms of ACC, PR, DR, and F1-scores, with only 7 features, is very satisfactory. Table 10 shows that our results are comparable to the results obtained by Sarhan et al. in [9] using 39 features with the ET model.

Tables 12 and 13 demonstrate a comparison of the validation results of multi-class and binary classification using the five machine-learning models with only seven features over the NF-CSE-CIC-IDS2018-v2 dataset. According to these two tables, we can also notice that using seven features was enough to predict each class (out of 15 attack and benign classes) in the NF-CSE-CIC-IDS2018-v2 dataset in the multi-classification (Table 12) and to differentiate between anomaly and begin instances in binary classification (Table 13) successfully. The proposed IDS showed outstanding performance for each class in terms of ACC, PR, DR, and

**Table 8**
Multi-classification for NF-UNSW-NB15-v2 dataset.

| Our Approaches | | | | | | | | | | | | | Sarhan et al. [9] | | Sayed et al. [37] | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Class | DT | | | RF | | | XGB | | | ET | | | ET | | CNN | | |
| | PR | DR | F1 | PR | DR | F1 | PR | DR | F1 | PR | DR | F1 | DR | F1 | PR | DR | F1 |
| Analysis | 07.57 | 16.44 | 10.37 | 06.76 | 08.75 | 07.62 | **09.11** | 12.67 | 10.60 | 08.03 | **16.89** | **10.89** | 30.89 | 17 | 88 | 93 | 16 |
| Backdoor | 27.91 | **13.32** | **18.04** | 27.78 | 13.18 | 17.87 | **43.51** | 09.81 | 16.01 | 28.12 | 13.18 | 17.95 | 40.30 | 18 | 11 | 67 | 19 |
| Benign | 99.81 | 99.79 | 99.80 | **99.82** | 99.80 | 99.81 | **99.85** | 99.80 | **99.82** | 99.81 | 99.80 | 99.81 | 99.85 | 100 | 100 | 97 | 99 |
| DoS | 29.62 | **28.60** | 29.10 | 32.10 | 26.86 | **29.25** | **33.42** | 20.55 | 25.45 | 30.03 | 27.82 | 28.88 | 29.57 | 36 | 22 | 37 | 28 |
| Exploits | 76.19 | 79.67 | 77.89 | 76.23 | 80.70 | 78.40 | 74.73 | **83.61** | **78.92** | **76.40** | 80.59 | 78.44 | 80.41 | 84 | 29 | 67 | 41 |
| Fuzzers | **78.20** | 78.45 | 78.32 | 75.84 | 80.13 | 77.93 | 68.53 | **83.64** | 75.33 | 78.18 | 78.88 | **78.53** | 80.57 | 85 | 70 | 85 | 77 |
| Generic | 87.97 | 81.94 | 84.85 | 86.29 | **82.44** | 84.32 | **93.22** | 75.97 | 83.72 | 88.36 | 82.00 | **85.06** | 85.15 | 90 | 30 | 70 | 42 |
| Reconnaissance | 86.45 | 77.70 | 81.84 | 80.82 | 78.68 | 79.74 | 84.11 | **79.18** | 81.57 | **86.53** | 77.75 | **81.91** | 80.02 | 83 | 69 | 95 | 80 |
| Shellcode | 55.88 | 56.44 | **56.16** | 54.89 | 56.93 | 55.89 | **58.42** | **42.08** | 48.92 | 55.37 | 56.19 | 55.77 | 87.67 | 69 | 15 | 60 | 23 |
| Worms | 69.23 | 52.94 | 60.00 | 70.73 | **56.86** | 63.04 | **72.50** | **56.86** | **63.74** | 71.79 | 54.90 | 62.22 | 85.98 | 69 | 19 | 100 | 31 |
| Weighted Avg | 98.79 | 98.73 | 98.75 | **98.75** | **98.76** | **98.75** | **98.77** | **98.76** | **98.73** | 98.80 | 98.75 | 98.77 | 98.90 | 99 | – | – | – |
| Accuracy | 98.73% | | | **98.76%** | | | **98.76%** | | | 98.76% | | | – | | | | |

**Table 9**
Bi-classification for NF-UNSW-NB15-v2 dataset.

| Class | DT | | | RF | | | XGB | | | NB | | | ET | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | PR | DR | F1 | PR | DR | F1 | PR | DR | F1 | PR | DR | F1 | PR | DR | F1 |
| Benign | 99.85 | 99.78 | 99.82 | 99.88 | 99.78 | 99.83 | **99.92** | 99.75 | **99.84** | 96.03 | **99.99** | 97.97 | 99.86 | 99.79 | 99.82 |
| Attack | 94.83 | 96.36 | 95.59 | 94.84 | 97.09 | 95.96 | 94.21 | **98.17** | **96.15** | 42.25 | 00.11 | 00.21 | **94.91** | 96.70 | 95.79 |
| Weighted Avg | 99.65 | 99.65 | 99.65 | 99.68 | 99.68 | 99.68 | **99.70** | **99.69** | **99.69** | 93.90 | 96.03 | 94.09 | 99.67 | 99.66 | 99.66 |
| Accuracy | 99.65% | | | 99.68% | | | **99.69%** | | | 96.03% | | | 99.66% | | |

**Table 10**
Multi-classification for NF-BoT-IoT-v2 dataset.

| Our Approaches | | | | | | | | | | | | | Sarhan et al. [9] | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Class | DT | | | RF | | | XGB | | | ET | | | ET | |
| | PR | DR | F1 | PR | DR | F1 | PR | DR | F1 | PR | DR | F1 | DR | F1 |
| Benign | 95.28 | 95.57 | 95.43 | **95.30** | 95.58 | **95.44** | **95.30** | 95.52 | 95.41 | 95.29 | **95.59** | **95.44** | 99.76 | 100 |
| DDoS | **99.40** | **99.43** | **99.42** | **99.40** | **99.43** | **99.42** | **99.40** | **99.43** | **99.42** | **99.40** | **99.43** | **99.42** | 99.99 | 100 |
| DoS | 98.42 | **98.33** | **98.38** | **98.43** | 98.32 | **98.38** | 98.42 | **98.33** | 98.37 | 98.42 | **98.33** | **98.38** | 99.99 | 100 |
| Reconnaissance | **93.30** | 93.64 | 93.47 | 93.22 | **93.73** | **93.48** | **93.30** | 93.61 | 93.45 | **93.30** | 93.64 | 93.47 | 99.93 | 100 |
| Theft | 71.19 | 30.20 | 42.41 | 69.69 | **31.32** | **43.22** | **75.29** | 27.39 | 40.16 | 71.76 | 30.34 | 42.62 | 83.01 | 85 |
| Weighted Avg | **98.52** | **98.52** | **98.52** | **98.53** | **98.52** | **98.52** | **98.52** | **98.52** | **98.52** | **98.52** | **98.52** | **98.52** | 99.99 | 100 |
| Accuracy | **98.52%** | | | **98.52%** | | | **98.52%** | | | **98.52%** | | | - | |

**Table 11**
Bi-classification for NF-BoT-IoT-v2 dataset.

| Class | DT | | | RF | | | XGB | | | NB | | | ET | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | PR | DR | F1 | PR | DR | F1 | PR | DR | F1 | PR | DR | F1 | PR | DR | F1 |
| Benign | 95.28 | 95.58 | 95.43 | 95.29 | 95.58 | **95.44** | 95.17 | 95.39 | 95.28 | **95.50** | 04.79 | 09.13 | 95.29 | **95.59** | 95.43 |
| Attack | **99.98** | 99.98 | **99.98** | **99.98** | 99.98 | **99.98** | **99.98** | 99.98 | **99.98** | 99.66 | **100** | 99.83 | **99.98** | 99.98 | **99.98** |
| Weighted Avg | **99.97** | **99.97** | **99.97** | **99.97** | **99.97** | **99.97** | **99.97** | **99.97** | **99.97** | 99.64 | 99.66 | 99.50 | **99.97** | **99.97** | **99.98** |
| Accuracy | **99.97%** | | | **99.97%** | | | **99.97%** | | | 99.66% | | | **99.97%** | | |

F1-scores compared to the results obtained by Sarhan et al. in [9] using 39 features with the ET model. The PR, DR, and F1-score of the Brute Force-Web, Brute Force-XSS, DDOS attack HOIC, Infiltration, and SQL Injection attacks have a significant improvement that overcomes Sarhan et al. model performance, despite the small number of samples. This means that the seven selected features are robust even against some NF-CSE-CIC-IDS2018-v2 attack classes with small samples.

For the NF-ToN-IoT-v2 dataset, a different experiment was conducted to determine the best set of features that results in the best traffic classification regardless of the number of features. The new experiment was executed using the Binary Arithmetic Optimizer Algorithm (BAOA). For multi-classification, the results show that by using only 14 features, the ML models achieve excellent performance with an accuracy of over 98%. The 14 selected features are: 'PROTOCOL', 'IN_PKTS', 'OUT_BYTES', 'TCP_FLAGS', 'CLIENT_TCP_FLAGS', 'SERVER_TCP_FLAGS', 'SHORTEST_FLOW_PKT', 'MAX_IP_PKT_LEN', 'RETRANSMITTED_OUT_PKTS', 'NUM_PKTS_1024_TO_1514_BYTES', 'TCP_WIN_MAX_IN', 'TCP_WIN_MAX_OUT', 'ICMP_TYPE', 'DNS_QUERY_ID'. For binary classification, the results show that by using only 12 features, the ML models achieve excellent performance with an accuracy of 99.99%. The 12 selected features are: 'PROTOCOL', 'IN_PKTS', 'TCP_FLAGS', 'SERVER_TCP_FLAGS', 'DURATION_OUT', 'MAX_TTL', 'MAX_IP_PKT_LEN', 'RETRANSMITTED_OUT_PKTS', 'DST_TO_SRC_AVG_THROUGHPUT', 'NUM_PKTS_1024_TO_1514_BYTES', 'TCP_WIN_MAX_IN', 'DNS_QUERY_ID'.

**Table 12**
Multi-classification for NF-CSE-CIC-IDS2018-v2 dataset.

| Our Approaches | | | | | | | | | | | | | Sarhan et al. [9] | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Class | DT | | | RF | | | XGB | | | ET | | | ET | |
| | PR | DR | F1 | PR | DR | F1 | PR | DR | F1 | PR | DR | F1 | DR | F1 |
| Benign | **99.50** | 99.98 | **99.74** | **99.50** | 99.98 | **99.74** | 99.49 | **99.99** | **99.74** | **99.50** | 99.98 | **99.74** | 99.69 | 100 |
| Bot | 99.96 | **99.95** | 99.95 | **99.99** | **99.95** | **99.97** | **99.99** | 99.93 | 99.96 | **99.99** | **99.95** | **99.97** | 100 | 100 |
| Brute Force -Web | 76.27 | **13.55** | 23.02 | 78.76 | 13.40 | 22.91 | **86.73** | 12.80 | 22.31 | 76.92 | **13.55** | **23.05** | 28.05 | 1 |
| Brute Force -XSS | **74.07** | 14.18 | 23.81 | 71.93 | **14.54** | **24.19** | 70.69 | **14.54** | 24.12 | 72.73 | 14.18 | 23.74 | 29.34 | 0 |
| DDOS attack-HOIC | **99.72** | **100** | **99.86** | **99.72** | **100** | **99.86** | **99.72** | **100** | **99.86** | **99.72** | **100** | **99.86** | 57.33 | 73 |
| DDOS attack-LOIC-UDP | 98.23 | 98.86 | 98.54 | 98.23 | 98.86 | 98.54 | **99.03** | **99.03** | **99.03** | 98.54 | 98.86 | 98.70 | 99.29 | 100 |
| DDoS attacks-LOIC-HTTP | 99.59 | **99.99** | **99.79** | **99.60** | **99.99** | **99.79** | **99.60** | **99.99** | **99.79** | **99.60** | **99.99** | **99.79** | 100 | 100 |
| DoS attacks-GoldenEye | **100** | **100** | **100** | **100** | **100** | **100** | **100** | **100** | **100** | **100** | **100** | **100** | 100 | 100 |
| DoS attacks-Hulk | **99.98** | **100** | **99.99** | **99.98** | **100** | **99.99** | **99.98** | **100** | **99.99** | **99.98** | **100** | **99.99** | 100 | 100 |
| DoS attacks-SlowHTTPTest | 99.98 | **100** | 99.99 | **100** | **100** | **100** | **100** | **100** | **100** | **100** | **100** | **100** | 100 | 100 |
| DoS attacks-Slowloris | **100** | **74.64** | **85.48** | **100** | **74.64** | **85.48** | **100** | **74.64** | **85.48** | **100** | **74.64** | **85.48** | 99.99 | 100 |
| FTP-BruteForce | **100** | **100** | **100** | **100** | **100** | **100** | **100** | **100** | **100** | **100** | **100** | **100** | 100 | 100 |
| Infilteration | 94.42 | 30.40 | 45.98 | 94.65 | **30.46** | **46.09** | **97.44** | 28.88 | 44.56 | 94.81 | 30.40 | 46.03 | 39.58 | 43 |
| SQL Injection | 78.57 | 08.89 | 14.77 | 87.50 | 10.37 | 18.54 | **100** | **11.85** | **21.19** | 92.31 | 08.89 | 16.22 | 41.44 | 0 |
| SSH-Bruteforce | **100** | **100** | **100** | **100** | **100** | **100** | **100** | **100** | **100** | **100** | **100** | **100** | 100 | 100 |
| Weighted Avg | **99.50** | **99.52** | **99.40** | **99.50** | **99.52** | **99.40** | **99.51** | **99.52** | **99.39** | **99.50** | **99.52** | **99.40** | 96.90 | 98 |
| Accuracy | **99.52%** | | | **99.52%** | | | **99.52%** | | | **99.52%** | | | – | |

**Table 13**
Bi-classification for NF-CSE-CIC-IDS2018-v2 dataset.

| Class | DT | | | RF | | | XGB | | | NB | | | ET | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | PR | DR | F1 | PR | DR | F1 | PR | DR | F1 | PR | DR | F1 | PR | DR | F1 |
| Benign | **99.50** | 99.98 | **99.74** | **99.50** | 99.98 | **99.74** | 99.49 | 99.98 | **99.74** | 88.06 | **100** | 93.65 | **99.50** | 99.98 | **99.74** |
| Attack | 99.84 | **96.29** | 98.03 | 99.85 | **96.29** | **98.04** | **99.86** | 96.23 | 98.01 | 00.00 | 00.00 | 00.00 | 99.85 | **96.29** | **98.04** |
| Weighted Avg | **99.54** | **99.54** | **99.54** | **99.54** | **99.54** | **99.54** | 99.54 | 99.53 | 99.53 | 77.54 | 88.06 | 82.46 | **99.54** | **99.54** | **99.54** |
| Accuracy | **99.54%** | | | **99.54%** | | | 99.53% | | | 88.06% | | | **99.54%** | | |

**Table 14**
Multi-classification for NF-ToN-IoT-v2 dataset.

| Our Approaches | | | | | | | | | | | | | Sarhan et al. [9] | | Awad et al. [31] | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Class | DT | | | RF | | | XGB | | | ET | | | ET | | RF | | |
| | PR | DR | F1 | PR | DR | F1 | PR | DR | F1 | PR | DR | F1 | DR | F1 | PR | DR | F1 |
| Benign | 99.37 | 99.72 | 99.54 | **99.39** | 99.72 | **99.55** | 99.18 | 99.60 | 99.39 | 99.38 | **99.73** | **99.55** | 99.44 | 99 | 100 | 100 | 100 |
| Backdoor | 99.64 | 99.73 | 99.68 | 99.74 | **99.75** | 99.74 | 99.96 | 99.73 | **99.84** | **99.70** | **99.75** | 99.72 | 99.79 | 100 | 100 | 100 | 100 |
| DDoS | 98.50 | 99.35 | 98.92 | **98.52** | 99.36 | **98.94** | 98.32 | 99.22 | 98.72 | 98.51 | **99.37** | **98.94** | 98.76 | 99 | 98 | 98 | 98 |
| DoS | 92.90 | 89.81 | **91.33** | 92.91 | 89.81 | **91.33** | **92.92** | 89.78 | 91.32 | 92.91 | **89.81** | **91.33** | 89.41 | 91 | 78 | 78 | 78 |
| Injection | 89.50 | 94.41 | 91.89 | 90.16 | 94.25 | **92.16** | **91.56** | 87.87 | 89.68 | 89.74 | **94.42** | 92.02 | 90.14 | 91 | 93 | 91 | 92 |
| MITM | 34.38 | 87.13 | 49.31 | 33.04 | **87.61** | 47.98 | **38.28** | 85.59 | **52.90** | 34.49 | 87.53 | 49.48 | 37.45 | 45 | 59 | 59 | 59 |
| Password | 96.85 | 97.38 | 97.11 | **96.98** | **97.39** | **97.18** | 96.52 | 93.73 | 95.10 | 96.91 | 97.38 | 97.14 | 97.16 | 97 | 97 | 97 | 97 |
| Ransomware | 96.79 | 94.99 | 95.88 | **97.46** | 95.29 | 96.36 | 97.42 | 94.49 | 95.93 | 96.94 | **96.10** | **96.52** | 97.29 | 98 | 99 | 99 | 99 |
| Scanning | 99.61 | 99.75 | **99.68** | 99.61 | **99.76** | **99.68** | 99.57 | 99.40 | 99.48 | **99.62** | 99.75 | **99.68** | 99.67 | 100 | 100 | 100 | 100 |
| XSS | **97.51** | 94.95 | 96.21 | 97.41 | 95.14 | **96.26** | 94.96 | **96.16** | 95.56 | 97.49 | 95.01 | 96.23 | 96.83 | 96 | 93 | 95 | 94 |
| Weighted Avg | 98.20 | 98.18 | 98.19 | **98.23** | **98.20** | **98.21** | 97.78 | 97.70 | 97.74 | 98.21 | 98.19 | 98.20 | 98.05 | 98 | 98 | 98 | 98 |
| Accuracy | 98.18% | | | **98.21%** | | | 97.72% | | | 98.20% | | | – | | 98% | | |

Tables 14 and 15 compare the validation results of multi-class and binary classification using the five machine-learning models with only 14 and 12 selected features respectively over the NF-ToN-IoT-v2 dataset. For the multi-classification, the RF model slightly outperforms the other machine-learning models with 98.21%, 98.23%, 98.20%, and 98.21% for ACC, PR, DR, and F1-score, respectively (Table 14). For the binary classification process, the ET slightly outperforms the other machine-learning models with 99.99% for ACC, PR, DR, and F1 (Table 15). Thus, according to these two tables, we can conclude that the 14 chosen features were enough to predict each class (out of 10 attack and benign classes) in the NF-ToN-IoT-v2 dataset in the multi-classification and the 12 chosen features were enough to differentiate between anomaly and begin instances in binary classification successfully.

Tables 14 and 15 show that our attack classification models achieved an almost perfect multi and binary classification performance (ACC, PR, DR, and F1-score). With only seven features, the obtained results were slightly better than those attained by Sarhan et al. in [9] using 39 features with the ET model and Awad et al. in [31] using 17 and 13 features for multi and binary classification, respectively. Also, the detection rate of the MITM class shows a significant improvement that overcomes the Sarhan et al. model's performance despite the small number of samples. This means that the 14 selected features for multi-classification (12 features for Binary classification) are robust for NF-ToN-IoT-v2 attack and benign classes regardless of the number of samples.

The two tables below summarize the validation results of the five ML models across the four datasets for the multi (Table 16) and binary (Table 17) classification. These results were acquired using only seven features from NF-CSE-CIC-IDS2018-v2, NF-UNSW-NB15-v2, and NF-BoT-IoT-v2 datasets and, 14 and 12 features in the case of NF-ToN-IoT-v2 dataset for the multi and binary classification, respectively.

**Table 15**
Bi-classification for NF-ToN-IoT-v2 dataset.

| Class | DT | | | RF | | | XGB | | | NB | | | ET | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | PR | DR | F1 | PR | DR | F1 | PR | DR | F1 | PR | DR | F1 | PR | DR | F1 |
| Benign | 99.37 | 99.72 | 99.54 | 99.42 | 99.71 | 99.56 | 99.26 | 99.44 | 99.35 | 46.18 | 99.99 | 63.18 | 99.99 | 99.99 | 99.99 |
| Attack | 99.84 | 99.64 | 99.74 | 99.83 | 99.67 | 99.75 | 99.68 | 99.58 | 99.63 | 01.13 | 00.01 | 00.02 | 99.99 | 99.99 | 99.99 |
| Weighted Avg | 99.75 | 99.38 | 99.56 | 99.68 | 99.68 | 99.68 | 99.52 | 99.20 | 99.36 | 17.34 | 35.99 | 23.40 | 99.99 | 99.99 | 99.99 |
| Accuracy | 99.69% | | | 99.77% | | | 99.62% | | | 40.80% | | | 99.99% | | |

**Table 16**
Multi-classification using BAOA selected features.

| Model | NF-ToN-IoT-v2 | | | | NF-UNSW-NB15-v2 | | | | NF-BoT-IoT-v2 | | | | NF-CSE-CIC-IDS2018-v2 | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | ACC | PR | DR | F1 | ACC | PR | DR | F1 | ACC | PR | DR | F1 | ACC | PR | DR | F1 |
| DT | 98.18 | 98.20 | 98.18 | 98.19 | 98.73 | 98.79 | 98.73 | 98.75 | **98.52** | **98.53** | **98.52** | **98.52** | **99.52** | 99.50 | **99.52** | **99.40** |
| RF | **98.21** | **98.23** | **98.20** | **98.21** | **98.76** | 98.75 | **98.76** | 98.75 | **98.52** | **98.53** | **98.52** | **98.52** | **99.52** | 99.50 | **99.52** | **99.40** |
| XGB | 97.72 | 97.78 | 97.70 | 97.74 | **98.76** | 98.77 | **98.76** | 98.73 | **98.52** | 98.52 | **98.52** | **98.52** | **99.52** | **99.51** | **99.52** | 99.39 |
| NB | 32.32 | 50.49 | 32.30 | 39.40 | 00.17 | 92.75 | 00.17 | 00.13 | 48.47 | 43.81 | 48.47 | 32.55 | 00.38 | 66.18 | 00.38 | 00.30 |
| ET | 98.20 | 98.21 | 98.19 | 98.20 | 98.75 | **98.80** | 98.75 | **98.77** | **98.52** | 98.52 | **98.52** | **98.52** | **99.52** | 99.50 | **99.52** | **99.40** |

**Table 17**
Bi-classification using BAOA selected features.

| Model | NF-ToN-IoT-v2 | | | | NF-UNSW-NB15-v2 | | | | NF-BoT-IoT-v2 | | | | NF-CSE-CIC-IDS2018-v2 | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | ACC | PR | DR | F1 | ACC | PR | DR | F1 | ACC | PR | DR | F1 | ACC | PR | DR | F1 |
| DT | 99.76 | 99.67 | 99.67 | 99.67 | 99.65 | 99.65 | 99.65 | 99.65 | 99.97 | 99.97 | 99.97 | 99.97 | **99.54** | **99.54** | **99.54** | **99.54** |
| RF | 99.77 | 99.68 | 99.68 | 99.68 | 99.66 | 99.68 | 99.66 | 99.68 | 99.97 | 99.97 | 99.97 | 99.97 | **99.54** | **99.54** | **99.54** | **99.54** |
| XGB | 99.62 | 99.52 | 99.20 | 99.36 | **99.69** | **99.70** | **99.69** | **99.69** | 99.97 | 99.97 | 99.97 | 99.97 | 99.53 | 99.54 | 99.53 | 98.53 |
| NB | 40.80 | 17.34 | 35.99 | 23.40 | 96.03 | 93.90 | 96.03 | 94.09 | 99.66 | 99.64 | 99.66 | 99.50 | 88.06 | 77.54 | 88.06 | 82.46 |
| ET | **99.99** | **99.99** | **99.99** | **99.99** | 99.66 | 99.67 | 99.66 | 99.66 | **99.98** | **99.98** | **99.98** | **99.98** | **99.54** | **99.54** | **99.54** | **99.54** |

According to Tables 16, and 17, we can see that by applying a subset of the features on all the NF datasets, we achieve very satisfactory performances in terms of ACC, PR, DR, and F1 scores. This means these selected features can be considered a standard feature set that would perform well over any NF-based dataset.

Moreover, according to these two tables, we can deduce that the tree-based machine learning algorithms (DT, RF, and ET) achieved the best multi-classification results scoring 98.21%, 98.76%, 98.52%, and 99.52% of classification accuracy for the aforementioned datasets, respectively. Also, these models achieved the highest binary classification, scoring 99.99%, 99.69%, 99.98%, and 99.54% classification accuracy for the, respectively, on the same datasets.

Another noteworthy observation is that the DT model results were similar to those of the RF and ET models. ET has the reputation of being a solid modeling technique and for being more robust than decision trees. However, it is more complex and computationally greedier. This means the use of DT speeds up the training time of the proposed IDS system while maintaining similar performance to ET, RF, and XGB.

Usually, ET results in higher performance than DT in the presence of noisy features. However, both ET and DT algorithms perform similarly when all the features are relevant. Thus, it is essential to note that the NF datasets are noiseless.

Tables 18 and 19 summarize the performance of the existing IDS systems stated in the literature and against our proposed IDS-based optimization and machine learning systems for multi (Table 18) and binary (Table 19) classification, respectively. The analysis of the reduction in training time is done regardless of the hardware used by defining the feature reduction rate, which is correlated to the training time.

Due to the recency of the utilized datasets, it is essential to highlight that these were the only reliable studies we could compare our work against. Our proposed system relies on NF-based features. Thus, we cannot compare our work against methods that utilized other datasets.

According to these tables, we can see that results obtained by our proposed IDS system achieved almost perfect multi and binary classification performance. The proposed system has shown significant improvements by achieving high prediction capabilities while utilizing a minimal number of features across all the NF datasets. In addition, the low FAR value means that the proposed IDS is able to correctly identify the attack classes and perfectly discriminate them from the benign class. Thus, the selected standard set of features can classify network traffic on all the NF datasets. Our IDS system achieves almost similar multi and binary classification performance (ACC, PR, DR, and F1) compared to the related works but with a smaller number of features, around 84% features reduction rate, compared to the 9% and 60% reduction rate of Sarhan et al. [9] and Awad et al. [31], respectively.

The results deem our system practical. The proposed NIDS achieved similar accuracy to those exhibited by recent work. Furthermore, the fact that these scores were obtained despite the significant reduction in the number of utilized features gives our proposed system a clear advantage over the current work. Additionally, our study has shown that the modified Arithmetic Optimization Algorithm greatly impacted the feature selection process, enhancing the NetFlow-based IDS's performance.

**Table 18**
Approaches that used v2 NF for multi-classification (FRR: Features Reduction Rate).

| Paper | Dataset | # Features | FRR | Model | Accuracy | Precision | Detection Rate | F1-score | FAR |
|---|---|---|---|---|---|---|---|---|---|
| Sarhan et al. [9] | NF-ToN-IoT-v2 | 39 | 9% | ET | – | – | 98.05% | 98% | – |
| | NF-UNSW-NB15-v2 | | | | – | – | 98.90% | 99% | – |
| | NF-BoT-IoT-v2 | | | | – | – | 99.99% | 100% | – |
| | NF-CSE-CIC-IDS2018-v2 | | | | – | – | 96.90% | 98% | – |
| Awad et al. [31] | NF-ToN-IoT-v2 | 17 | 60% | RF | 98% | 98% | 98% | 98% | – |
| Our | NF-ToN-IoT-v2 | 14 | 67% | DT/RF/XGB/ET | 98.21% | 98.23% | 98.20% | 98.21% | 0.22% |
| | NF-ToN-IoT-v2 | 7 | 84% | | 97.21% | 97.15% | 97.17% | 97.16% | 0.32% |
| | NF-UNSW-NB15-v2 | | | | 98.76% | 98.80% | 98.76% | 98.77% | 3.71% |
| | NF-BoT-IoT-v2 | | | | 98.52% | 98.53% | 98.52% | 98.52% | 4.4% |
| | NF-CSE-CIC-IDS2018-v2 | | | | 99.52% | 99.51% | 99.52% | 99.40% | 3.27% |

**Table 19**
Approaches that used v2 NF for binary-classification (FRR: Features Reduction Rate).

| Paper | Dataset | # Features | FRR | Model | Accuracy | Precision | Detection Rate | F1-score | FAR |
|---|---|---|---|---|---|---|---|---|---|
| Karanfilovska et al. [29] | NF-ToN-IoT-v2 10% of it | 43 (with identifiers) | 0% | XGBoost | 98.80% | 98.80% | 98.80% | 99% | – |
| Sarhan et al. [9] | NF-ToN-IoT-v2 | 39 | 9% | ET | 99.64% | – | 99.76% | 100% | – |
| | NF-UNSW-NB15-v2 | | | | 99.73% | – | 97.07% | 97% | – |
| | NF-BoT-IoT-v2 | | | | 100% | – | 100% | 100% | – |
| | NF-CSE-CIC-IDS2018-v2 | | | | 99.35% | – | 96.89% | 97% | |
| Awad et al. [31] | NF-ToN-IoT-v2 | 13 | 70% | RF | 100% | 100% | 100% | 100% | – |
| Our | NF-ToN-IoT-v2 | 12 | 72% | DT/RF/XGB/ET | 99.99% | 99.99% | 99.99% | 99.99% | 0.02% |
| | NF-ToN-IoT-v2 | 7 | 84% | | 99.99% | 99.99% | 99.99% | 99.99% | 0.02% |
| | NF-UNSW-NB15-v2 | | | | 99.69% | 99.70% | 99.69% | 99.69% | 4.08% |
| | NF-BoT-IoT-v2 | | | | 99.98% | 99.98% | 99.98% | 99.98% | 0.86% |
| | NF-CSE-CIC-IDS2018-v2 | | | | 99.54% | 99.54% | 99.54% | 99.54% | 3.26% |

## 5. Conclusion and future work

Utilizing a predefined standard set of features across various IoT datasets can be very beneficial. Various datasets have different features, making it challenging to propose a universal model that can accurately classify attacks and independently identify patterns representing each attack. A standard feature set allows for consistent and comparable network traffic analysis, making identifying and classifying different types of attacks easier. This is important for network security as it enables more effective detection and response to potential threats. The NetFlow standard has various features, and if dataset creators adopt it as a feature representation method, it could serve as a standard. Such a standard would allow researchers to accurately assess their models and fairly compare their performance against others. However, such a standard must consist of the fewest possible features. A low number of features suggests lower computational cost, faster prediction time, and less storage space.

This research proposes a NIDS for large-scale IoT NetFlow-based networks and defines a standard NetFlow feature set for multi-classification and binary classification. This subset of features is extracted from a NetFlow-based dataset, which is later applied to other NetFlow datasets. To acquire this subset of features, we experimented with and modified several well-known optimization algorithms, such as the Arithmetic Optimization Algorithm (AOA). The results achieved by the modified AOA were the highest, where only seven out of 43 NetFlow features were selected to represent the different attack patterns for binary and multi-class classification. The same seven features were used to classify the network traffic in other NetFlow datasets (NF-CSE-CIC-IDS2018-v2, NF-ToN-IoT-v2, and NF-UNSW-NB15-v2) for multi and binary classification attack patterns using several machine learning models.

The findings concluded that the results achieved by the tree-based algorithms were the highest and similar to the state-of-the-art studies that utilized a much larger set of features. These results were attained after reducing the dataset features by up to 84%, making them even more remarkable.

The results show that the proposed system is practical for detecting intrusions. The proposed NIDS achieved similar accuracy to the literature studies, with up to 99% and 98% accuracy for binary and multi-classification, respectively. These scores were achieved despite decreasing the number of utilized features by up to 84% compared to the 9% reduction rate in the literature. It was discovered that the use of the modified Arithmetic Optimization Algorithm had a high impact on the feature selection process, significantly impacting the performance of NetFlow-based IDS. However, the proposed system's main limitation is the problem type, which only applies to IDS datasets similar to those used in this study. The modified version of AOA is also used for the binary version of the search space. Also, the AOA can get stuck in a premature convergence due to the unsuitable balance between exploration and exploitation during the search. Another dilemma is the computational time, where at each iteration of AOA, we need to execute the ML method, which requires extensive computational time. Therefore, in the future, the proposed method can be improved and validated by trying other datasets and running the system in a parallel environment to improve the computational time. Furthermore, the proposed AOA can be hybridized with other efficient components to empower the balance between exploration and exploitation, thus improving the final outcomes.

While the feature standardization concept is novel, the authors recognize that applying the standard features to existing non-NF-based datasets may not be practical and that implementing it in real-life scenarios may be limited. Therefore, there is a need for further research to enhance such limitations of NIDS systems. The authors intend to investigate the role of optimization algorithms in

intrusion detection across various datasets. This investigation will explore different optimization algorithms across various features (NF and others) and identify the optimal set among them. This exploration will provide insights into the problem from a different perspective and offer more clarity to developing NIDS systems.

## Ethical approval

We further confirm that any aspect of the work covered in this manuscript that has involved human patients has been conducted with the ethical approval of all relevant bodies and that such approvals are acknowledged within the manuscript.

## Intellectual property

We confirm that we have given due consideration to the protection of intellectual property associated with this work and that there are no impediments to publication, including the timing of publication, with respect to intellectual property. In so doing we confirm that we have followed the regulations of our institutions concerning intellectual property.

## Declaration of competing interest

We wish to confirm that there are no known conflicts of interest associated with this publication and there has been no significant financial support for this work that could have influenced its outcome.

## Data availability

No data was used for the research described in the article.

## References

[1] The growth in connected IoT devices is expected to generate 79.4zb of data in 2025, according to a new IDC forecast, 2019, https://www.businesswire.com/news/home/20190618005012/en/The-Growth-in-Connected-IoT-Devices-is-Expected-to-Generate-79.4ZB-of-Data-in-2025-According-to-a-New-IDC-Forecast. (Accessed May 2022).
[2] K. Rose, S. Eldridge, L. Chapin, The internet of things: An overview, Internet Soc. (ISOC) 80 (2015) 1–50.
[3] P. Radanliev, D. De Roure, P. Burnap, O. Santos, Epistemological equation for analysing uncontrollable states in complex systems: Quantifying cyber risks from the internet of things, Rev. Socionetwork Strateg. 15 (2) (2021) 381–411.
[4] N. Martindale, M. Ismail, D.A. Talbert, Ensemble-based online machine learning algorithms for network intrusion detection systems using streaming data, Information 11 (6) (2020).
[5] K. Rajasekaran, K. Nirmala, Classification and importance of intrusion detection system, Int. J. Comput. Sci. Inf. Secur. 10 (8) (2012) 44.
[6] P. Garcia-Teodoro, J. Diaz-Verdejo, G. Maciá-Fernández, E. Vázquez, Anomaly-based network intrusion detection: Techniques, systems and challenges, Comput. Secur. 28 (1–2) (2009) 18–28.
[7] J. Hussain, S. Lalmuanawma, L. Chhakchhuak, A two-stage hybrid classification technique for network intrusion detection system, Int. J. Comput. Intell. Syst. 9 (5) (2016) 863–875.
[8] N.F. Haq, A.R. Onik, M.A.K. Hridoy, M. Rafni, F.M. Shah, D.M. Farid, Application of machine learning approaches in intrusion detection system: A survey, IJARAI-Int. J. Adv. Res. Artif. Intell. 4 (3) (2015) 9–18.
[9] M. Sarhan, S. Layeghy, M. Portmann, Towards a standard feature set for network intrusion detection system datasets, Mob. Netw. Appl. 27 (1) (2022) 357–370.
[10] NetFlow version 9 flow-record format, 2011, https://www.cisco.com/en/US/technologies/tk648/tk362/technologies_white_paper09186a00800a3db9.html. (Accessed March 2022).
[11] N.O. Leslie, Using semi-supervised learning for flow-based network intrusion detection, Cell 202 (2018) 528–0770.
[12] M. Sarhan, S. Layeghy, M. Portmann, Feature analysis for ML-based IIoT intrusion detection, 2021.
[13] J. Li, K. Cheng, S. Wang, F. Morstatter, R.P. Trevino, J. Tang, H. Liu, Feature selection: A data perspective, ACM Comput. Surv. 50 (6) (2017) 1–45.
[14] Q.M. Alzubi, M. Anbar, Z.N. Alqattan, M.A. Al-Betar, R. Abdullah, Intrusion detection system based on a modified binary grey wolf optimisation, Neural Comput. Appl. 32 (10) (2020) 6125–6137.
[15] Q.M. Alzubi, M. Anbar, Y. Sanjalawe, M.A. Al-Betar, R. Abdullah, Intrusion detection system based on hybridizing a modified binary grey wolf optimization and particle swarm optimization, Expert Syst. Appl. 204 (2022) 117597.
[16] N. Dash, S. Chakravarty, S. Satpathy, An improved harmony search based extreme learning machine for intrusion detection system, Mater. Today: Proc. (2021).
[17] D.H. Wolpert, W.G. Macready, No free lunch theorems for optimization, IEEE Trans. Evol. Comput. 1 (1) (1997) 67–82.
[18] L. Abualigah, A. Diabat, S. Mirjalili, M. Abd Elaziz, A.H. Gandomi, The arithmetic optimization algorithm, Comput. Methods Appl. Mech. Engrg. 376 (2021) 113609.
[19] J.O. Agushaka, A.E. Ezugwu, Advanced arithmetic optimization algorithm for solving mechanical engineering design problems, PLoS One 16 (8) (2021) e0255703.
[20] R.A. Ibrahim, L. Abualigah, A.A. Ewees, M.A. Al-Qaness, D. Yousri, S. Alshathri, M. Abd Elaziz, An electric fish-based arithmetic optimization algorithm for feature selection, Entropy 23 (9) (2021) 1189.
[21] M. Abd Elaziz, L. Abualigah, R.A. Ibrahim, I. Attiya, IoT workflow scheduling using intelligent arithmetic optimization algorithm in fog computing, Comput. Intell. Neurosci. 2021 (2021).
[22] M. Braik, A. Hammouri, J. Atwan, M.A. Al-Betar, M.A. Awadallah, White shark optimizer: A novel bio-inspired meta-heuristic algorithm for global optimization problems, Knowl.-Based Syst. 243 (2022) 108457.
[23] S. Mirjalili, S.M. Mirjalili, A. Lewis, Grey wolf optimizer, Adv. Eng. Softw. 69 (2014) 46–61.
[24] X.-S. Yang, A.H. Gandomi, Bat algorithm: A novel approach for global engineering optimization, Eng. Comput. (2012).
[25] S. Chen, G.I. Webb, L. Liu, X. Ma, A novel selective naïve Bayes algorithm, Knowl.-Based Syst. 192 (2020) 105361.

[26] Y. Liu, Y. Wang, J. Zhang, New machine learning algorithm: Random forest, in: International Conference on Information Computing and Applications, Springer, 2012, pp. 246–252.
[27] M. Brijain, R. Patel, M. Kushik, K. Rana, A survey on decision tree algorithm for classification, 2014.
[28] T. Chen, T. He, M. Benesty, V. Khotilovich, Y. Tang, H. Cho, K. Chen, et al., Xgboost: Extreme gradient boosting, R Package Version 0.4-2 1 (4) (2015) 1–4.
[29] M. Karanfilovska, T. Kochovska, Z. Todorov, A. Cholakoska, G. Jakimovski, D. Efnusheva, Analysis and modelling of a ML-based NIDS for IoT networks, Procedia Comput. Sci. 204 (2022) 187–195.
[30] M. Komisarek, M. Pawlicki, R. Kozik, W. Hołubowicz, M. Choraś, How to effectively collect and process network data for intrusion detection? Entropy 23 (11) (2021).
[31] M. Awad, S. Fraihat, K. Salameh, A. Al Redhaei, Examining the suitability of NetFlow features in detecting IoT network intrusions, Sensors 22 (16) (2022).
[32] N. Moustafa, J. Slay, UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set), in: 2015 Military Communications and Information Systems Conference, MilCIS, 2015, pp. 1–6.
[33] N. Koroniotis, N. Moustafa, E. Sitnikova, B.P. Turnbull, Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-IoT dataset, Future Gener. Comput. Syst. 100 (2019) 779–796.
[34] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, A. Anwar, TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems, IEEE Access 8 (2020) 165130–165150.
[35] I. Sharafaldin, A.H. Lashkari, A.A. Ghorbani, Toward generating a new intrusion detection dataset and intrusion traffic characterization, in: Proceedings of the 4th International Conference on Information Systems Security and Privacy, ICISSP, SciTePress, INSTICC, 2018, pp. 108–116.
[36] M. Sarhan, S. Layeghy, N. Moustafa, M. Gallagher, M. Portmann, Feature extraction for machine learning-based intrusion detection in IoT networks, 2021, arXiv preprint arXiv:2108.12722.
[37] N. Sayed, M. Shoaib, W. Ahmed, S. Qasem, A. Albarrak, F. Saeed, Augmenting IoT intrusion detection system performance using deep neural network, Comput. Mater. Contin. 74 (1) (2022) 1351–1374.
[38] T.-T.-H. Le, H. Kim, H. Kang, H. Kim, Classification and explanation for intrusion detection system based on ensemble trees and SHAP method, Sensors 22 (3) (2022) 1154.
[39] R. Younisse, A. Ahmad, Q. Abu Al-Haija, Explaining intrusion detection-based convolutional neural networks using Shapley additive explanations (SHAP), Big Data Cogn. Comput. 6 (4) (2022).
[40] M. Sarhan, S. Layeghy, M. Portmann, An explainable machine learning-based network intrusion detection system for enabling generalisability in securing IoT networks, 2021, arXiv e-prints. arXiv–2104.
[41] A. Basahel, M. Yamin, S. Basahel, E. Laxmi Lydia, Enhanced coyote optimization with deep learning based cloud-intrusion detection system, Comput. Mater. Contin. 74 (2) (2023) 4319–4336.
[42] R. Alkanhel, E.-S. El-Kenawy, A. Abdelhamid, A. Ibrahim, M. Alohali, M. Abotaleb, D. Khafaga, Network intrusion detection based on feature selection and hybrid metaheuristic optimization, Comput. Mater. Contin. 74 (2) (2023) 2677–2693.
[43] R. Alkanhel, D. Khafaga, E.-S. El-Kenawy, A. Abdelhamid, A. Ibrahim, R. Amin, M. Abotaleb, B. El-Den, Hybrid grey wolf and dipper throated optimization in network intrusion detection systems, Comput. Mater. Contin. 74 (2) (2023) 2695–2709.
[44] D. Khafaga, F. Karim, A. Abdelhamid, E.-S. El-Kenawy, H. Alkahtani, N. Khodadadi, M. Hadwan, A. Ibrahim, Voting classifier and metaheuristic optimization for network intrusion detection, Comput. Mater. Contin. 74 (2) (2023) 3183–3198.
[45] S. Vanitha, P. Balasubramanie, Improved ant colony optimization and machine learning based ensemble intrusion detection model, Intell. Autom. Soft Comput. 36 (1) (2023) 849–864.
[46] M. Alazab, R. Khurma, A. Awajan, D. Camacho, A new intrusion detection system based on Moth–Flame Optimizer algorithm, Expert Syst. Appl. 210 (2022).
[47] S. Sokkalingam, R. Ramakrishnan, An intelligent intrusion detection system for distributed denial of service attacks: A support vector machine with hybrid optimization algorithm based approach, Concurr. Comput.: Pract. Exper. 34 (27) (2022).
[48] C. Prajisha, A. Vasudevan, An efficient intrusion detection system for MQTT-IoT using enhanced chaotic salp swarm algorithm and LightGBM, Int. J. Inf. Secur. 21 (6) (2022) 1263–1282.
[49] W. Al-Yaseen, A. Idrees, F. Almasoudy, Wrapper feature selection method based differential evolution and extreme learning machine for intrusion detection system, Pattern Recognit. 132 (2022).
[50] R. Kumar, A. Malik, V. Ranga, An intellectual intrusion detection system using hybrid hunger games search and Remora optimization algorithm for IoT wireless networks, Knowl.-Based Syst. 256 (2022).
[51] H. Xu, Y. Lu, Q. Guo, Application of improved butterfly optimization algorithm combined with black widow optimization in feature selection of network intrusion detection, Electronics (Switzerland) 11 (21) (2022).
[52] S. Ethala, A. Kumarappan, A hybrid spider monkey and hierarchical particle swarm optimization approach for intrusion detection on internet of things, Sensors 22 (21) (2022).
[53] A. Dahou, M. Abd Elaziz, S.A. Chelloug, M.A. Awadallah, M.A. Al-Betar, M.A. Al-qaness, A. Forestiero, Intrusion detection system for IoT based on deep learning and modified reptile search algorithm, Comput. Intell. Neurosci. 2022 (2022).
[54] M. Aziz, A. Alfoudi, Feature selection of the anomaly network intrusion detection based on restoration particle swarm optimization, Int. J. Intell. Eng. Syst. 15 (5) (2022) 592–600.
[55] N. Kunhare, R. Tiwari, J. Dhar, Intrusion detection system using hybrid classifiers with meta-heuristic algorithms for the optimization and feature selection by genetic algorithm, Comput. Electr. Eng. 103 (2022).
[56] M. Imran, S. Khan, H. Hlavacs, F. Khan, S. Anwar, Intrusion detection in networks using cuckoo search optimization, Soft Comput. 26 (20) (2022) 10651–10663.
[57] M. Alweshah, S. Alkhalaileh, M. Beseiso, M. Almiani, S. Abdullah, Intrusion detection for IoT based on a hybrid shuffled shepherd optimization algorithm, J. Supercomput. 78 (10) (2022) 12278–12309.
[58] M. Ramkumar, T. Daniya, P. Mano Paul, S. Rajakumar, Intrusion detection using optimized ensemble classification in fog computing paradigm, Knowl.-Based Syst. 252 (2022).
[59] S. Mohamed, O. Alsaif, I. Saleh, Intrusion detection network attacks based on whale optimization algorithm, Ingenierie Des Systemes D'Information 27 (3) (2022) 441–446.
[60] S. Kareem, R. Mostafa, F. Hashim, H. El-Bakry, An effective feature selection model using hybrid metaheuristic algorithms for IoT intrusion detection, Sensors 22 (4) (2022).
[61] M. Otair, O. Ibrahim, L. Abualigah, M. Altalhi, P. Sumari, An enhanced Grey Wolf Optimizer based particle swarm optimizer for intrusion detection system in wireless sensor networks, Wirel. Netw. 28 (2) (2022) 721–744.
[62] T.D. Nguyen, M.-H. Shih, D. Srivastava, S. Tirthapura, B. Xu, Stratified random sampling from streaming and stored data, Distrib. Parallel Databases 39 (3) (2021) 665–710.
[63] J.R. Quinlan, C4. 5: Programs for Machine Learning, Elsevier, 2014.

[64] R. Primartha, B.A. Tama, Anomaly detection using random forest: A performance revisited, in: 2017 International Conference on Data and Software Engineering, ICoDSE, IEEE, 2017, pp. 1–6.
[65] A. Huč, J. Šalej, M. Trebar, Analysis of machine learning algorithms for anomaly detection on edge devices, Sensors 21 (14) (2021) 4946.
[66] P. Biswas, T. Samanta, Anomaly detection using ensemble random forest in wireless sensor network, Int. J. Inf. Technol. 13 (5) (2021) 2043–2052.
[67] S. Seifert, Application of random forest based approaches to surface-enhanced Raman scattering data, Sci. Rep. 10 (1) (2020) 1–11.
[68] A.P. Wibawa, A.C. Kurniawan, D.M.P. Murti, R.P. Adiperkasa, S.M. Putra, S.A. Kurniawan, Y.R. Nugraha, Naïve Bayes classifier for journal quartile classification, Int. J. Recent Contrib. Eng. Sci. IT 7 (2) (2019) 91–99.
[69] Netflow v2 features, 2021, https://cloudstor.aarnet.edu.au/plus/apps/onlyoffice/s/Y4tLFbVjWthpVKd?fileId=5240171798. (Accessed March 2022).
[70] D.M. Powers, Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation, 2020, arXiv preprint arXiv:2010.16061.